Real-Time Threat Detection and Incident Analysis Using Endpoint Telemetry in Enterprise Environments.

by

Aarif Rahman

Industry-sponsored Project

Submitted in partial fulfilment of B. Tech
(Information System)
degree programme

ABSTRACT

Cybersecurity has become an essential part of modern enterprise environments due to the increasing number of cyber threats and attacks. Organizations today rely heavily on digital systems, and any security breach can lead to data loss, financial impact, and operational disruption. As a result, continuous monitoring and protection of IT infrastructure has become very important.

One of the key areas in cybersecurity is endpoint security. Endpoint devices such as desktops, laptops, and servers are widely used in organizations and act as major entry points for cyber-attacks. These systems generate large amounts of data in the form of logs, known as endpoint telemetry, which can be analyzed to understand system behavior and detect potential threats.

Real-time threat detection plays an important role in identifying suspicious activities at an early stage and preventing security incidents. By analyzing endpoint telemetry using tools like SIEM, it becomes possible to detect abnormal activities, generate alerts, and perform incident analysis effectively. This project focuses on using endpoint telemetry to simulate a real-world security monitoring environment and understand how threats are detected and managed in enterprise systems.

Technologies In today's organizations, endpoints such as desktops and servers are widely used and act as major entry points for cyber-attacks. With the increase in cyber-threats, relying only on traditional antivirus solutions is not enough. Modern security approaches focus more on endpoint telemetry, where logs and system activities are continuously monitored to identify suspicious behavior.

In real-world environments, tools like Symantec Endpoint Protection (SEP) and SIEM platforms are used to detect and manage threats. However, challenges like large volume of alerts, false positives, and delayed response still exist. This creates the need for better methods to detect threats quickly and analyze incidents effectively.
This dissertation is based on studying how endpoint data can be used along with SIEM tools to improve real-time threat detection and incident analysis in a practical and controlled environment.

Contents

Module 1
INTRODUCTION

1.1 Background of the Study.

In recent years, the rapid advancement of digital technologies has significantly increased the reliance of organizations on IT systems and infrastructure. While this transformation has improved operational performance, it has also expanded the attack surface, leading to a rise in cybersecurity concerns. Modern cyber threats, including malicious software, phishing campaigns, ransomware attacks, and unauthorized system intrusions, are becoming more frequent and complex in enterprise environments.

Traditional security solutions, such as firewalls and antivirus software, primarily rely on signature-based detection methods, which are limited to identifying known attack patterns. As a result, they often fail to detect sophisticated and emerging threats, such as zero-day exploits and advanced persistent threats.

To address these challenges, organizations are increasingly adopting advanced monitoring approaches, including endpoint telemetry. This technique involves capturing real-time system-level data such as event logs, process activities, and network traffic from endpoint devices. The collected data enhances visibility into system behavior and supports early identification of abnormal activities.

Furthermore, the integration of endpoint telemetry with Security Information and Event Management (SIEM) platforms, such as Microsoft Sentinel, allows organizations to identify and investigate potential threats in real time. This approach improves threat detection capabilities and enables quicker response actions, ultimately strengthening the overall security posture of the organization.

1.2 Problem Statement.

In enterprise environments, detecting cybersecurity threats in real time remains a major challenge. Traditional security systems often fail to provide complete visibility into endpoint activities, making it difficult to identify suspicious behavior early. Moreover, these systems are highly dependent on predefined signatures, which limits their ability to detect unknown or evolving threats.

Another challenge is the large volume of log data generated by systems, which makes manual analysis inefficient and time-consuming. Security teams often struggle to correlate events and identify potential threats quickly.

Therefore, there is a need for a system that can collect endpoint telemetry data, analyze it effectively, and detect potential threats in real time using advanced techniques.

1.3 Need for the Project.

The motivation for this project comes from the growing need for more effective cybersecurity solutions that can identify threats as they occur. With the increasing complexity of cyberattacks, many modern threats are able to evade conventional security systems, which makes it necessary to use more advanced monitoring and detection techniques.

This project is designed to address the following requirements:

- Better visibility into activities happening on endpoint systems
- Early identification of abnormal or suspicious behavior
- Faster response to security incidents
- Centralized collection and analysis of system logs
- Capability to identify both known attacks and newly emerging threats

By implementing a system based on endpoint telemetry integrated with SIEM tools, it becomes easier for organizations to monitor system behavior and react to potential threats in a more efficient manner.

1.4 Objectives of Project.

The main objectives of this project are as follows:

- To study and understand the concept of endpoint telemetry in cybersecurity.
- To collect detailed system logs using monitoring tools such as Sysmon.
- To integrate collected logs into a SIEM platform (Microsoft Sentinel).
- To design and implement detection rules for identifying suspicious activities.
- To analyze logs and generate alerts in real time.
- To simulate attack scenarios and evaluate detection performance.
- To improve threat detection capabilities using behavior-based analysis.

1.5 Scope of the Study.

The scope of this project is focused on designing and implementing a real-time threat detection system using endpoint telemetry within a controlled lab environment. The system is limited to monitoring activities on a Windows / Windows Server-based endpoint and analyzing logs using Microsoft Sentinel.

The project includes:

- Collection of endpoint logs using Sysmon.
- Integration of logs with SIEM (Microsoft Sentinel).
- Development of detection rules using KQL.
- Simulation of basic attack scenarios such as PowerShell execution.
- Analysis of detection results.

However, the project does not cover large-scale enterprise deployment or advanced machine learning-based detection techniques. The focus is primarily on building a functional prototype for demonstration purposes.

Module 2
LITERATURE REVIEW

2.1 Overview of Cyber Threat Landscape.

In recent years, the cybersecurity environment has changed rapidly due to increased digital transformation and the widespread use of cloud-based platforms and enterprise systems. As organizations continue to adopt these technologies, cyber threats have also become more complex and difficult to detect using conventional security methods.

Various types of cyberattacks are commonly observed, including malware infections, phishing attempts, ransomware incidents, insider misuse, and advanced persistent threats. These threats are often designed in a way that allows them to avoid detection by traditional security controls and remain active in systems for longer durations.

Attackers are also using more advanced techniques such as social engineering, fileless attacks, and methods that misuse legitimate system tools to carry out malicious activities. Because of this, organizations require better monitoring solutions that can provide a detailed view of what is happening inside their systems.

The increasing frequency and sophistication of these attacks highlight the importance of real-time monitoring systems that can detect suspicious activities at an early stage and help in preventing major security incidents.

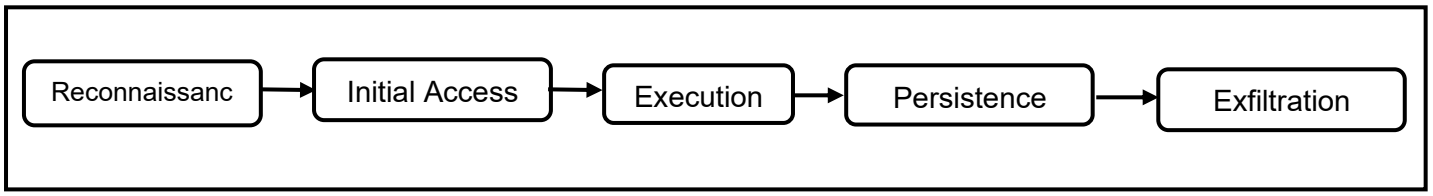

Figure 2.1: Cyber Attack Lifecycle

2.2 Endpoint Telemetry in Cybersecurity.

Endpoint telemetry can be understood as the continuous collection of system-level information from devices such as desktops, laptops, and servers. This information typically includes details related to running processes, file activities, registry updates, and network communication.

In the context of cybersecurity, endpoint telemetry is very important because it provides a detailed view of how systems are behaving. Unlike traditional security approaches that depend on limited log data, telemetry allows security analysts to monitor activities across the system more effectively.

By examining this data, it becomes easier to identify unusual patterns that may indicate potential threats. For instance, unexpected process execution or abnormal network traffic can act as early warning signs of malicious activity.

In many enterprise environments, endpoint telemetry is integrated with centralized monitoring platforms. This integration helps in identifying threats quickly and supports faster response actions when a security incident occurs.

2.3 Security Information and Event Management (SIEM).

Security Information and Event Management (SIEM) systems are designed to collect, store, and analyze log data from multiple sources within an organization. These sources include endpoints, servers, network devices, and applications.

SIEM platforms provide centralized visibility into security events and allow organizations to correlate logs from different systems. This helps in detecting complex attack patterns that may not be visible when analyzing individual logs.

Modern SIEM solutions such as Microsoft Sentinel offer cloud-based capabilities, enabling real-time data analysis and automated alert generation. These systems use query languages and rule-based detection techniques to identify potential threats.

SIEM tools also assist in incident response by providing detailed information about security events, helping security teams investigate and respond to incidents more efficiently.

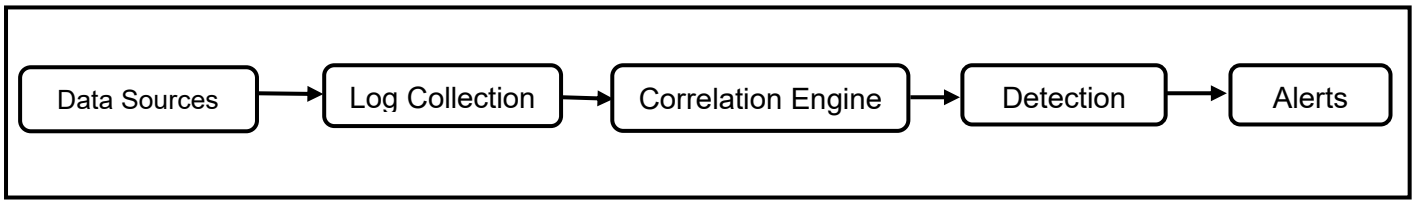

Figure 2.2: SIEM Workflow Architecture.

2.4 Sysmon for Endpoint Monitoring.

Sysmon (System Monitor) is a Windows system service that enhances the default logging capabilities of the operating system. It captures detailed system-level events and records them in the Windows Event Log.

Sysmon provides information such as process creation, network connections, changes in files, and updated made to the system registry. This level of detail is very useful for detecting suspicious activities which could be a sign of a possible cyberattack.

A major advantage of Sysmon is that it can generate high-quality logs that can be used for forensic analysis and threat detection. It operates continuously in the background and records system events in real time.

When integrated with SIEM tools, Sysmon logs can be analyzed to detect malicious behavior patterns and generate alerts for suspicious activities.

2.5 Threat Detection Techniques:

Threat detection techniques are used to identify malicious activities within a system. These techniques vary in approach and effectiveness depending on the nature of the threat being identified.

2.5.1 Signature-Based Detection.

Signature-based detection relies on predefined patterns or signatures of known threats. These signatures are stored in databases and used to identify malicious files or activities.

This method is widely used in antivirus systems and intrusion detection systems.

It is highly effective for identifying known threats in a fast and reliable manner.

On the other hand, signature-based detection has an important limitation. It cannot detect unknown or new attacks that are not recognized by existing signatures. As a result, this method is less effective in dealing with zero-day attacks and advanced threats.

2.5.2 Anomaly-Based Detection.

Anomaly-based detection works by identifying changes from normal system behavior. It creates a baseline of regular activity and marks any unusual patterns as possible threats.

This method is helpful for identifying previously unknown attacks, such as zero-day vulnerabilities. It does not rely on predefined signatures and can identify previously unseen threats.

However, anomaly-based detection often generates a large number of false alerts. Regular changes in system activity can sometimes be mistaken for suspicious activities. Therefore, proper tuning and baseline configuration are required to improve accuracy

2.5.3 Behavior-Based Detection.

Behavior-based detection analyzes system activity and identifies suspicious behavior patterns commonly associated with cyberattacks. It focuses on what actions are being performed instead of depending only on predefined signatures.

This technique is effective for detecting advanced threats, for example fileless malware attacks and techniques that misuse built-in system tools. It can identify malicious activity even if the specific attack pattern is not previously known.

Behavior-based detection provides a balance between accuracy as well as flexibility, which makes it appropriate for modern cybersecurity systems. It is widely used in SIEM along with endpoint detection and response (EDR) systems.

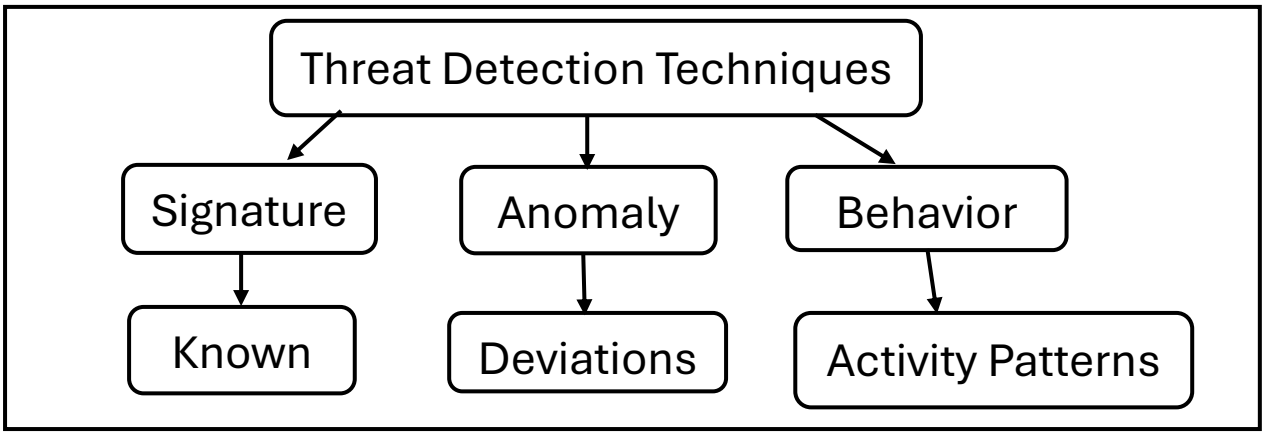

Figure 2.3: Classification of Threat Detection Techniques.

2.6 Comparative Analysis of Detection Techniques.

Different detection techniques have different advantages and drawbacks. Signature-based detection remains effective for previously identified threats but lacks the ability to detect new attacks. Anomaly-based detection can identify unknown threats but may produce false alerts.

Behavior-based detection offers a more balanced approach by analyzing system activities and identifying suspicious patterns. It combines the advantages of both methods while reducing their limitations.

In this project, behavior-based detection is considered the most suitable approach because it provides accurate and real-time detection capabilities.

2.7 Challenges in Threat Detection Systems.

Despite advancements in cybersecurity technologies, several challenges exist in threat detection systems. One major challenge is the large amount of log data produced by various systems. Analyzing this data efficiently requires powerful tools and optimized algorithms.

Another challenge is the generation of incorrect alerts, in which normal regular activities are marked as potential threats. Such issues can increase the workload on security teams and reduce the effectiveness of detection systems.

Additionally, attackers use advanced evasion techniques to bypass security systems. These techniques include encryption, obfuscation, and the use of legitimate system tools.

Managing these challenges requires continuous improvement in detection techniques and system configurations.

2.8 Research Gap Identification.

The literature review highlights that while many studies focus on threat detection techniques, there is a lack of practical implementation combining endpoint telemetry and SIEM tools in real-time environments.

Most existing systems rely either on theoretical models or large-scale enterprise solutions, which may not be feasible for academic or small-scale implementations.

There is also limited focus on demonstrating how telemetry data can be used to detect specific attack scenarios using practical tools such as Sysmon and Microsoft Sentinel.

This project aims to address these gaps by developing a practical system that integrates endpoint telemetry with SIEM and demonstrates real-time threat detection in a controlled environment.

2.9 Summary.

This chapter reviewed key concepts related to cybersecurity, including the evolving threat landscape, endpoint telemetry, SIEM systems, and different threat detection techniques.

The analysis shows that traditional detection methods are insufficient for modern threats, and there is a need for more advanced approaches such as behavior-based detection.

The literature also highlights the importance of real-time monitoring and centralized log analysis in identifying and handling cyber threats in an efficient way.

The findings from this chapter provide the foundation for designing the proposed system, which is discussed in subsequent chapters.

Module 3
METHODOLOGY

3.1 Research Approach.

The research for this project follows a practical and experimental approach to design and implement a real-time threat detection system. The study is focused on understanding how endpoint telemetry can be used to monitor system activities and detect potential threats.

The approach includes both theoretical learning and hands-on implementation. Initially, the concepts of endpoint telemetry, SIEM systems, and threat detection techniques were studied through existing research and documentation. Based on this understanding, a practical system was developed in a controlled lab environment.

The research follows a step-by-step process that includes:

- Identifying the problem
- Designing the system architecture
- Setting up the environment
- Collecting and analyzing logs
- Implementing detection mechanisms

This method makes sure the system is not just only conceptually correct but is also practically functional.

3.2 System Design Methodology.

The system is designed using a modular architecture approach, where each component performs a specific function. This makes the system easy to understand, implement, and extend.

The design consists of the following layers:

- Endpoint Layer – Generates system activity data.
- Logging Layer – Captures logs using Sysmon.
- Collection Layer – Transfers logs to SIEM.
- Analysis Layer – Processes logs using Microsoft Sentinel.
- Detection Layer – Applies rules to identify threats.

The system design follows a top-down approach, where the overall architecture is defined first, and then individual components are developed and integrated.

This methodology ensures:

- Proper separation of components.
- Easy troubleshooting.
- Better scalability.

3.3 Data Collection Strategy.

The data collection approach is based on capturing detailed endpoint activity using Sysmon.

Sysmon is configured to capture important system events including:

- Process creation events
- Network connections
- File changes
- Registry changes

The collected logs are saved in the Windows Event Log and later sent to the SIEM platform for further analysis.

The data collection strategy focuses on:

- Capturing relevant security events
- Filtering unnecessary logs
- Ensuring continuous monitoring

This approach helps in collecting meaningful data required for threat detection without overloading the system.

3.4 Log Analysis Approach.

The log analysis process is carried out using Microsoft Sentinel, which acts as a centralized platform for monitoring and analyzing logs.

The analysis approach involves:

- Collecting logs from endpoint systems
- Storing logs in the SIEM workspace
- Using Kusto Query Language (KQL) to analyze logs
- Identifying suspicious activities based on patterns

Logs are analyzed based on key parameters such as:

- Process name
- Command-line arguments
- Event ID
- Network activity

The system focuses on identifying abnormal behavior such as:

- Execution of suspicious commands
- Unauthorized access attempts
- Unusual process activity

This structured approach ensures effective identification of potential threats.

3.5 Detection Rule Design Strategy.

The detection strategy is based on behavior-based rule implementation, where system activities are analyzed to identify suspicious patterns.

Detection rules are created using KQL queries in Microsoft Sentinel. These rules are designed to detect:

- Suspicious PowerShell execution
- Unknown process activity
- Unusual network connections

Example detection rule logic:

- Monitor process creation events
- Filter specific commands (e.g., PowerShell)
- Generate alerts for suspicious behavior

The detection rules are designed with the following objectives:

- High accuracy
- Real-time detection
- Reduced false positives

The system uses simple yet effective rules to demonstrate real-time threat detection.

3.6 Tools Selection Justification.

The tools used in this project are selected based on their effectiveness, availability, and compatibility with the system.

Sysmon

- Provides detailed endpoint logging
- Captures system-level events
- Lightweight and easy to configure

Microsoft Sentinel

- Cloud-based SIEM solution
- Enables real-time monitoring and analysis
- Supports advanced query-based detection

Virtual Machine

- Creates a safe environment for testing
- Allows attack simulation without risk

Kusto Query Language (KQL)

- Powerful language query for log analysis
- Used for detection rule implementation

The selection of these tools ensures that the system is practical, scalable, and aligned with real-world cybersecurity practices.

3.7 Workflow Overview.

The complete workflow of this system is made to perform real-time threat detection using endpoint telemetry.

Step-by-step workflow:

1. Endpoint system generates activity (process execution, network events).
2. Sysmon captures these activities and logs them.
3. Logs are stored in the system and forwarded to SIEM.
4. Microsoft Sentinel collects and processes the logs.
5. Detection rules analyze the logs using KQL queries.
6. Alerts are generated for suspicious activities.
7. Security analysis is performed based on alerts.

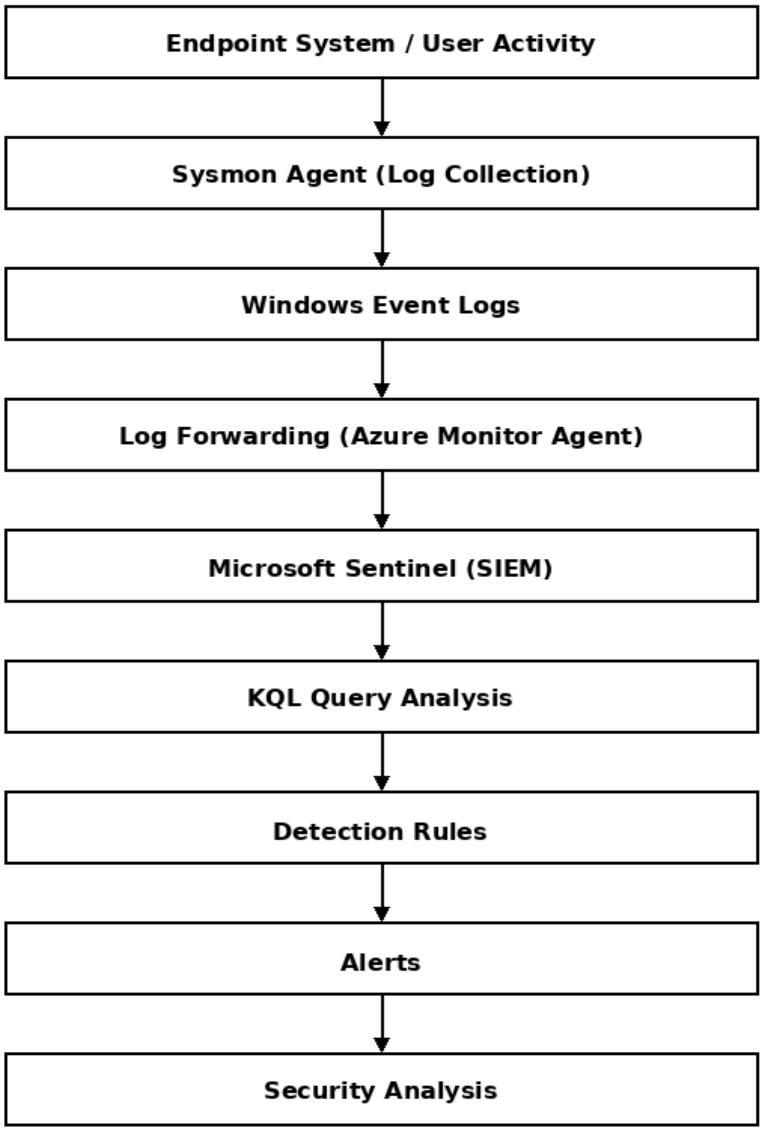

Figure 3.1: Proposed System Workflow.

Module 4
SYSTEM ARCHITECTURE & DESIGN

4.1 Overview of Proposed System.

The proposed system is designed to perform real-time threat detection using endpoint telemetry data. The system focuses on collecting system-level logs from endpoint devices, processing these logs using a centralized SIEM platform, and detecting suspicious activities based on predefined rules.

The architecture is designed to simulate an enterprise security monitoring environment where endpoint activities are continuously monitored and analyzed. The system integrates multiple components such as Sysmon for log collection and Microsoft Sentinel for log analysis.

The main goal of the architecture is to provide:

- Continuous monitoring of endpoint activity.
- Centralized log management.
- Real-time threat detection.
- Alert generation for suspicious activities.

The system is structured in a layered format to ensure modularity and ease of implementation.

4.2 Architecture Diagram Description.

The system architecture consists of multiple interconnected components that work together to enable real-time threat detection.

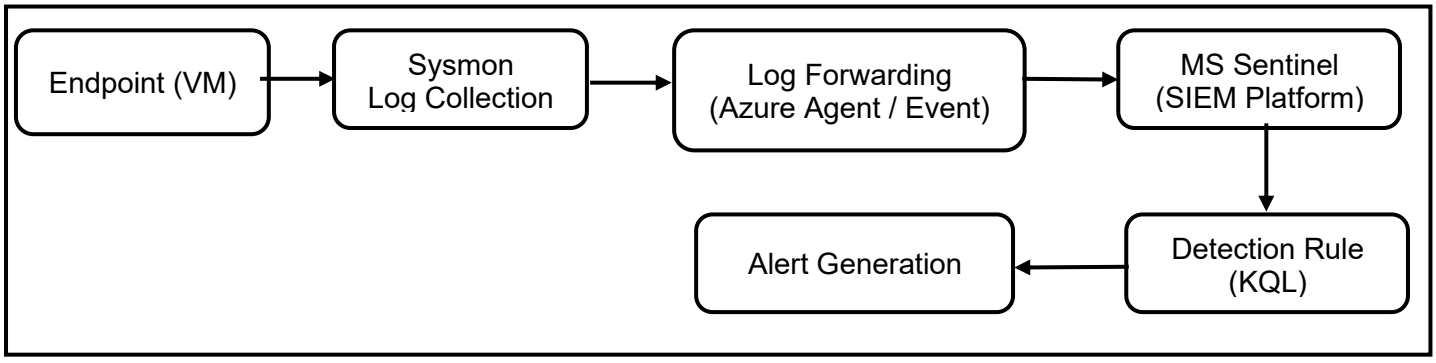

Figure 4.1: System Architecture Diagram.

The architecture follows a pipeline model:

Endpoint System → Sysmon → Log Collection → Microsoft Sentinel → Detection Rules →Alerts

Each component in the architecture performs a specific function:

- Endpoint generates activity data
- Sysmon captures detailed logs
- Logs are sent to the SIEM platform
- Detection rules analyze logs
- Alerts are generated for suspicious behavior

This structured flow ensures efficient data processing and timely detection of threats.

4.3 Data Flow Diagram (DFD).

The data flow within the system describes how information moves from endpoint devices to the detection system.

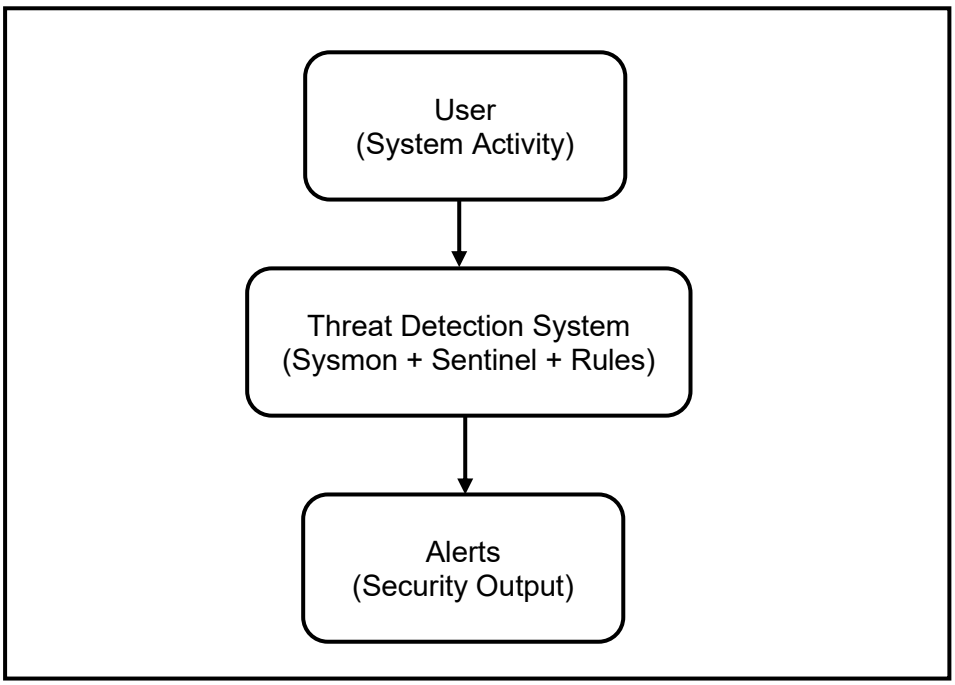

Figure 4.2: Data Flow Diagram Level 0.

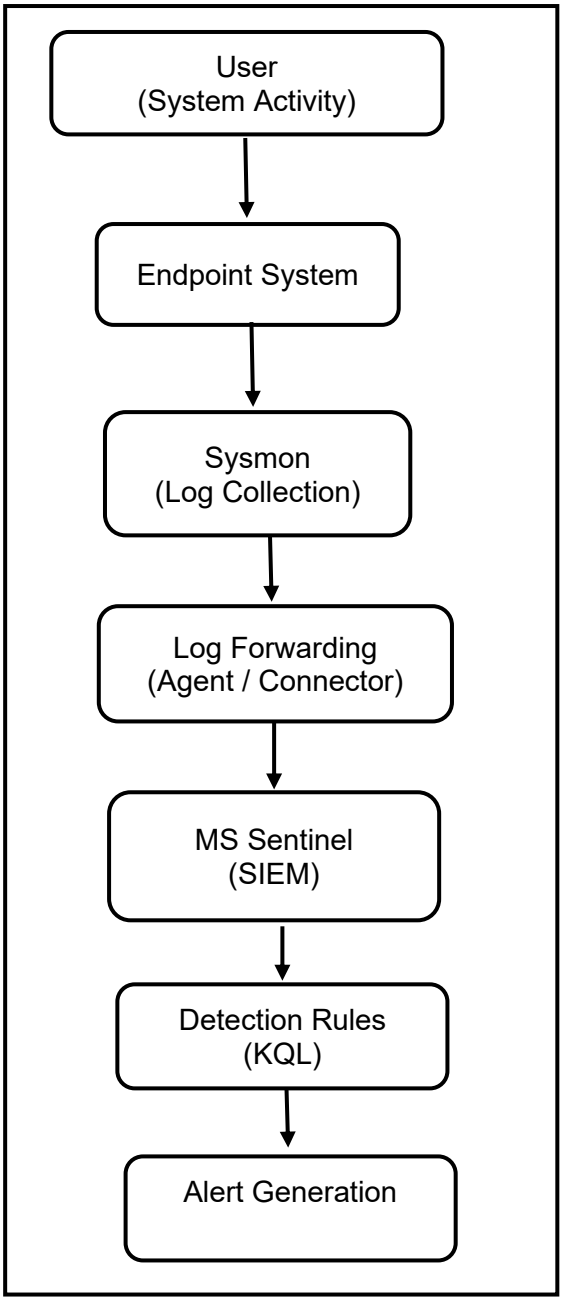

Figure 4.3: Data Flow Diagram Level 1.

Data Flow Steps:

- Endpoint generates activity (process execution, network requests).
- Sysmon collects and logs system events.
- Logs are forwarded to the SIEM platform.
- Microsoft Sentinel stores and processes log data.
- Queries analyze logs for suspicious patterns.
- Alerts are generated based on detection rules.

The data flow ensures smooth movement of information between components and enables real-time analysis.

4.4 System Components.

The proposed system is divided into multiple layers, each responsible for a specific function.

4.4.1 Endpoint Layer.

The endpoint layer represents the system where activities are performed.
It includes:

- User actions
- Application processes
- Network communication

This layer acts as the source of data, generating events that are monitored by the system.

4.4.2 Log Collection Layer.

The log collection layer captures system-level events using Sysmon. It records detailed logs including:

- Process creation events
- Network connections
- File modifications

This layer ensures that all relevant system activities are recorded for analysis.

4.4.3 SIEM Layer.

The SIEM layer is responsible for centralized log storage and analysis. Microsoft Sentinel is used in this project for:

- Collecting logs from endpoints
- Storing logs in the cloud
- Providing query-based analysis

This layer acts as the core analysis engine of the system.

4.4.4 Detection Layer.

The detection layer applies rules and queries to identify suspicious activities.
It includes:

- KQL-based detection rules
- Alert generation mechanisms

This layer analyzes system behavior and identifies potential threats in real time.

4.5 Role of Sysmon in Architecture.

Sysmon plays a important role in the proposed architecture by giving detailed endpoint telemetry data. It enhances the default logging capabilities of the operating system and captures high-quality logs required for threat detection.

Sysmon is responsible for:

- Monitoring process execution
- Tracking network connections
- Logging system events in real time

These logs form the foundation for detecting suspicious activities. Without Sysmon, the system would not have sufficient visibility into endpoint behavior.

4.6 Integration with Microsoft Sentinel.

Microsoft Sentinel acts as the central platform for log analysis and threat detection. It receives logs collected by Sysmon and processes them using built-in monitoring capabilities.

The integration involves:

- Configuring data connectors
- Sending logs to Sentinel workspace
- Running KQL queries on collected data

Sentinel enables:

- Real-time log monitoring
- Detection rule implementation
- Alert generation

This integration allows the system to function as a complete SIEM-based detection solution.

4.7 Advantages of Proposed System.

The proposed system offers several advantages:

- **Real-Time Monitoring:** Continuous tracking of endpoint activities
- **Centralized Log Analysis:** All logs are analyzed at a single platform
- **Improved Visibility:** Detailed insight into system behavior
- **Early Threat Detection:** Identification of suspicious activities at early stages
- **Scalability:** System can be expanded for larger environments
- **Practical Implementation:** Uses real-world tools like Sysmon and Sentinel

Module 5
TOOLS AND TECHNOLOGIES

5.1 Overview of Tools Used.

This project uses a combination of monitoring, analysis, and virtualization tools to implement a real-time threat detection system. The tools were selected to simulate a real-world enterprise security environment and to provide practical exposure to cybersecurity technologies.

The primary tools used in this project include:

- Sysmon for endpoint telemetry collection.
- Microsoft Sentinel as the SIEM platform.
- Virtual Machine for environment setup.
- Windows Operating System for endpoint simulation.
- Kusto Query Language (KQL) for log analysis.

These tools work together to collect, process, and analyze system-level data, enabling real-time detection of suspicious activities.

5.2 Sysmon (Working & Configuration).

Sysmon is a Windows system service that provides detailed information about system activity. It enhances the default logging capabilities of the Windows operating system by capturing granular event data.

Key functionalities of Sysmon:

- Monitoring process creation events.
- Tracking network connections.
- Logging file and registry changes.
- Recording system activity in real time.

Sysmon generates logs that are stored in the Windows Event Log. These logs contain valuable information required for threat detection and forensic analysis.

Why is Sysmon used:

- Provides detailed endpoint telemetry.
- Helps identify suspicious behavior patterns.
- Works efficiently in the background.
- Integrates easily with SIEM tools.

Sysmon plays a crucial role in this project as it acts as the primary data source for detecting threats.

5.3 Microsoft Sentinel (SIEM).

Microsoft Sentinel is a cloud-based Security Information and Event Management (SIEM) solution that collects and analyzes log data from multiple sources.

Key features of Microsoft Sentinel:

- Centralized log collection and monitoring.
- Real-time data analysis.
- Query-based detection using KQL.
- Alert generation for suspicious activities.
- Integration with multiple data sources.

In this project, Microsoft Sentinel is used to:

- Collect logs generated by Sysmon.
- Store and manage telemetry data.
- Analyze logs using queries.
- Detect and alert suspicious activities.

The use of Sentinel allows the system to simulate an enterprise-level security monitoring environment.

5.4 Kusto Query Language (KQL).

Kusto Query Language (KQL) is a powerful query language used for analyzing log data in Microsoft Sentinel.

Key capabilities of KQL:

- Filtering log data based on conditions
- Searching for specific events
- Identifying patterns in system activity
- Creating detection queries

Example Use in Project:
KQL queries are used to detect suspicious activities such as:

- Execution of PowerShell commands
- Unauthorized system processes
- Abnormal network activity

Example query:

```
SecurityEvent
| where EventID == 4688
| where Process contains "powershell"
```

KQL plays an important role in implementing detection rules and alerts in the system.

5.5 Virtual Machine Setup.

A virtual machine is used in this project to create a controlled and isolated testing environment. This allows safe execution of simulated attacks without affecting the host system.

Advantages of using VM:

- Provides a safe environment for testing.
- Allows easy configuration and reset.
- Supports multiple operating systems.
- Enables simulation of real-world scenarios.

In this project, the virtual machine is used to:

- Install Windows OS
- Run Sysmon for log collection
- Perform attack simulations

The use of a VM helps in ensuring that the system can be tested without causing any real damage.

5.6 Windows Operating System.

The Windows operating system is used as the endpoint system in this project. It is chosen because most enterprise environments rely on Windows-based systems, making it relevant for cybersecurity testing.

Role of Windows OS:

- Acts as the endpoint generating system activity
- Provides Event Viewer for log storage
- Supports Sysmon installation

The Windows environment allows simulation of real-world activities such as process execution and network communication, which are essential for telemetry-based monitoring

5.7 Technology Stack Summary.

The following table summarizes the tools and technologies used in the project:

Tool / Technology	**Purpose**
Sysmon	Collect endpoint telemetry data
Microsoft Sentinel	Log analysis and threat detection
KQL	Querying and detection rules
Virtual Machine	Safe testing environment
Windows OS	Endpoint simulation

Table 5.1: Tools Summary.

Module 6
SETUP PHASE

6.1 System Requirements.

To implement the proposed threat detection system, certain hardware and software requirements were identified. These requirements ensure proper functioning of the tools and smooth execution of the system.

6.1.1 Hardware Requirements.

The following hardware configuration was used for the project:

- Processor: Intel Core i5 or equivalent
- RAM: Minimum 8 GB
- Storage: At least 50 GB of free disk space
- Internet Connectivity: Required for cloud-based services

This configuration ensures smooth performance of the virtual machine and other tools used in the system.

6.1.2 Software Requirements.

The project requires the following software components:

- Windows Operating System (for endpoint simulation).
- Sysmon (System Monitor tool).
- Microsoft Azure Account.
- Microsoft Sentinel (SIEM platform).
- Virtual Machine software (VMware / VirtualBox).
- Internet browser (for cloud access).

These tools are essential for setting up the environment, collecting logs, and performing threat detection.

6.2 Environment Setup.

The environment setup is an important step in the implementation of the system. In this project, a virtual machine (VM) is used to simulate an endpoint system.

6.2.1 Virtual Machine Configuration.

A virtual machine was created using virtualization software to provide a controlled and isolated environment for testing.

The following steps were performed:

- Installation of VM software
- Creation of a new virtual machine
- Allocation of system resources (RAM, CPU, storage)
- Installation of Windows operating system

The use of a virtual machine ensures that any test or simulated attack does not affect the host system.

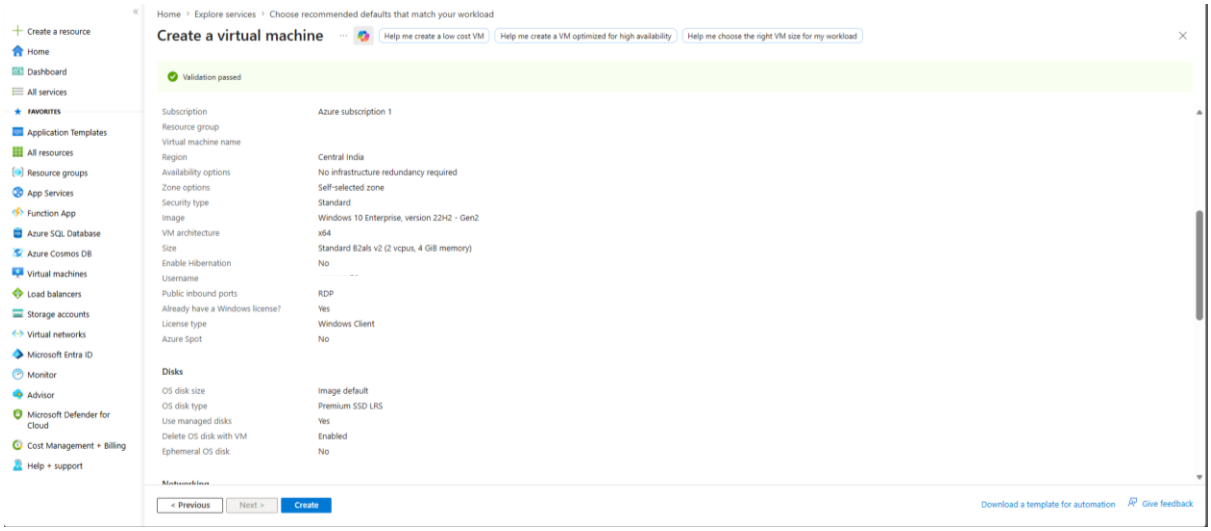

Figure 6.1: Azure VM Overview.

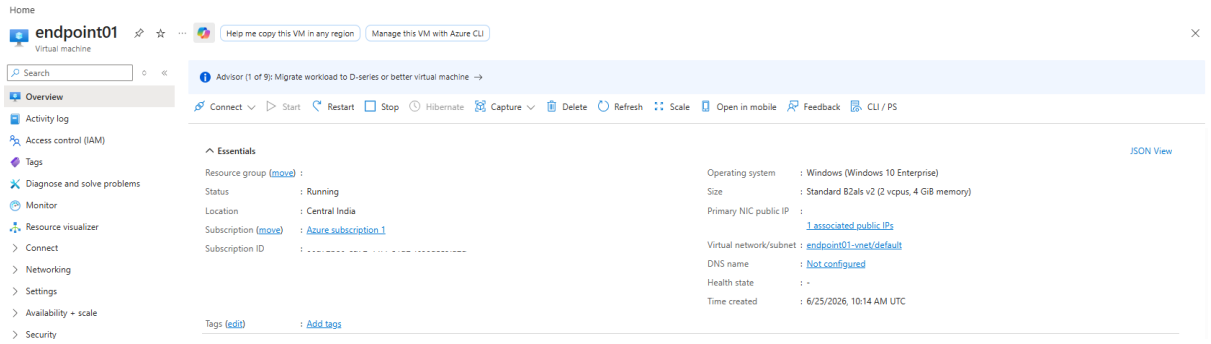

Figure 6.2: Azure VM Running.

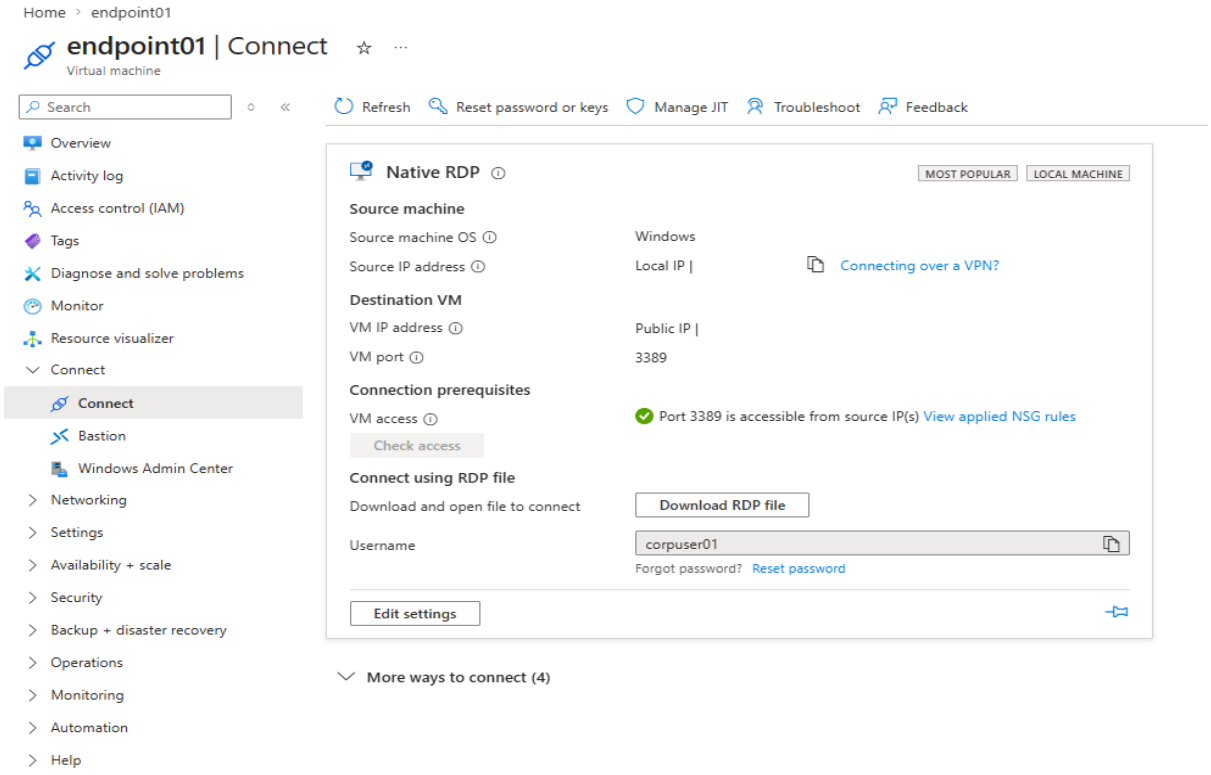

Figure 6.3: RDP Connection to Azure VM.

6.2.2 Operating System Setup.

A Windows operating system was installed inside the virtual machine. The system was configured to enable logging and monitoring features required for the project.

Key configurations include:

- Enabling Windows Event Logging.
- Updating system settings.
- Verifying system performance.

The Windows system acts as the endpoint where all activities are monitored.

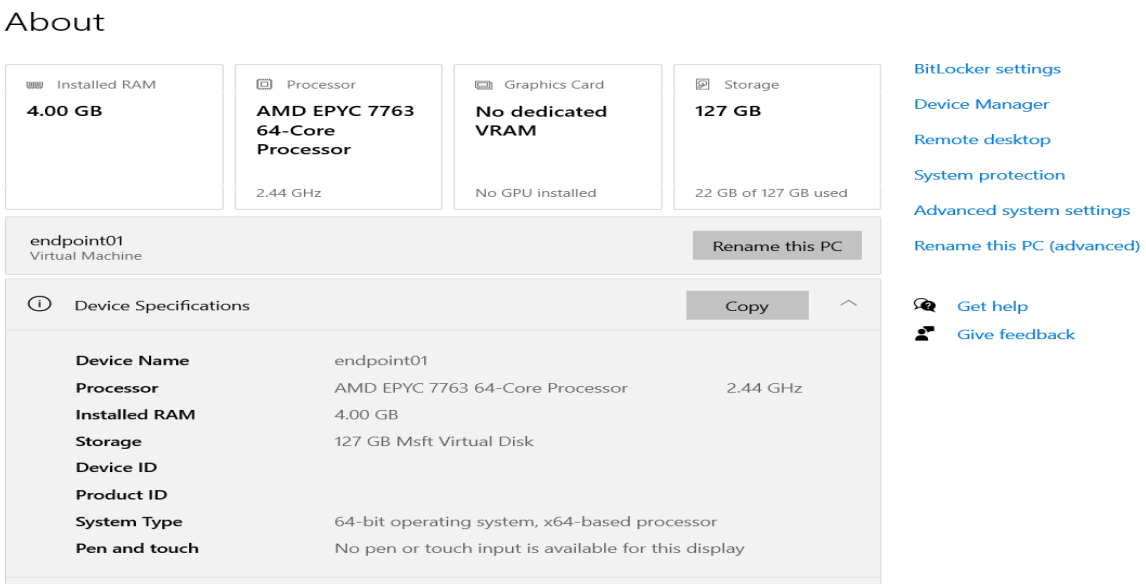

Figure 6.4: OS Machine Overview.

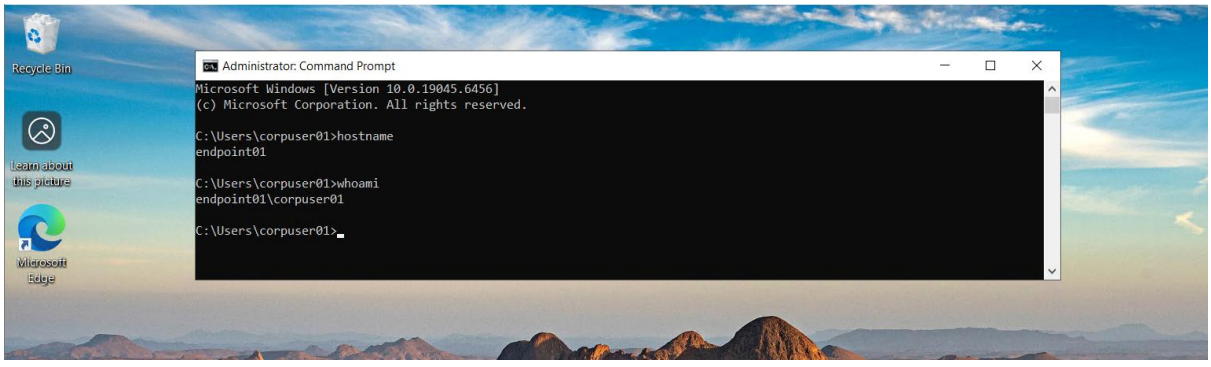

Figure 6.5: OS Machine Running.

6.3 Sysmon Installation and Configuration.

Sysmon was installed on the endpoint system to collect detailed telemetry data.

Installation Steps:

- Download Sysmon from official Microsoft source
- Extract files and run installation command
- Apply configuration file

Example installation command:

sysmon -i sysmonconfig.xml
sysmon64.exe -accepteula -i sysmonconfig-export.xml

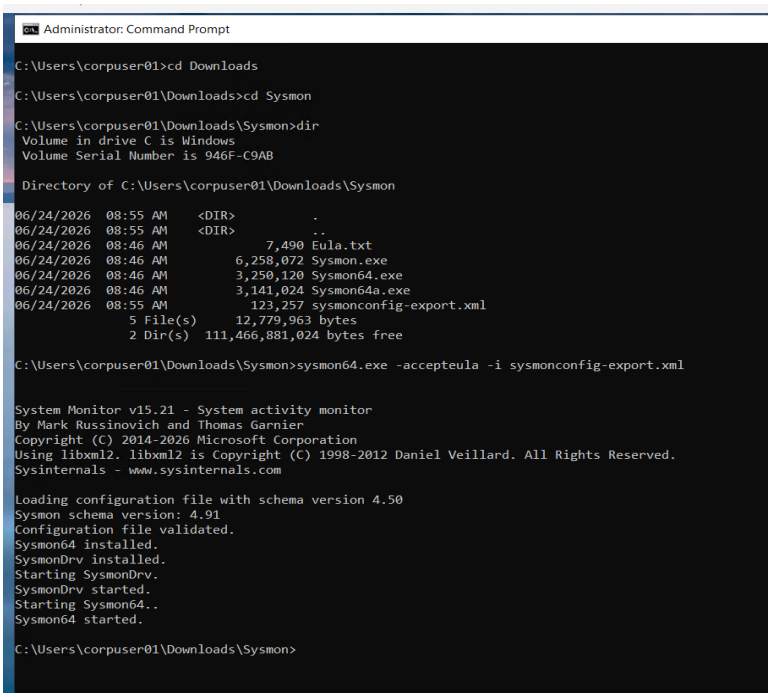

Figure 6.6: Sysmon Installation Command Execution.

Configuration

Sysmon is configured to capture important system events such as:

- Process creation
- Network connections
- File changes

The configuration file defines which events should be logged. Proper configuration ensures that only relevant data is collected.

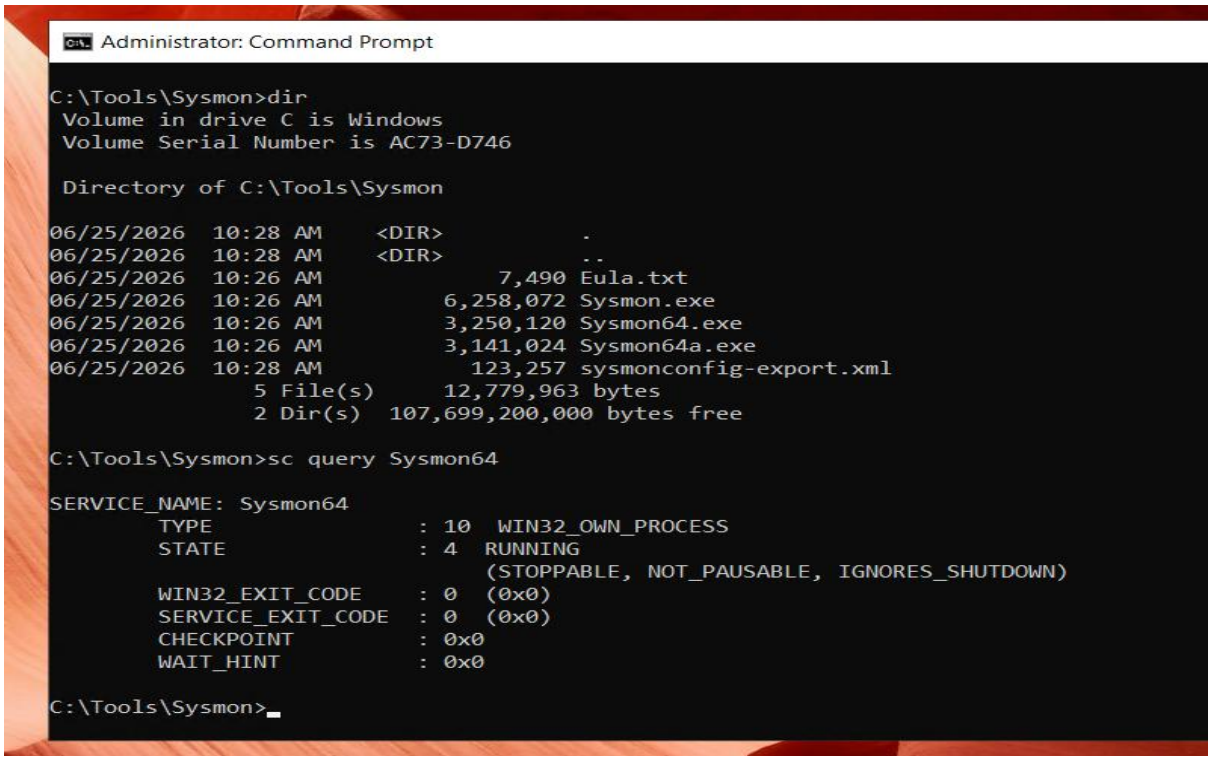

Figure 6.7: Sysmon Configuration and Running.

6.4 Log Generation and Testing.

After installing Sysmon, system activities were performed to generate logs. These activities include:

- Opening applications.
- Running commands (e.g., PowerShell).
- Accessing network resources.

The logs generated by Sysmon were verified using the Windows Event Viewer.

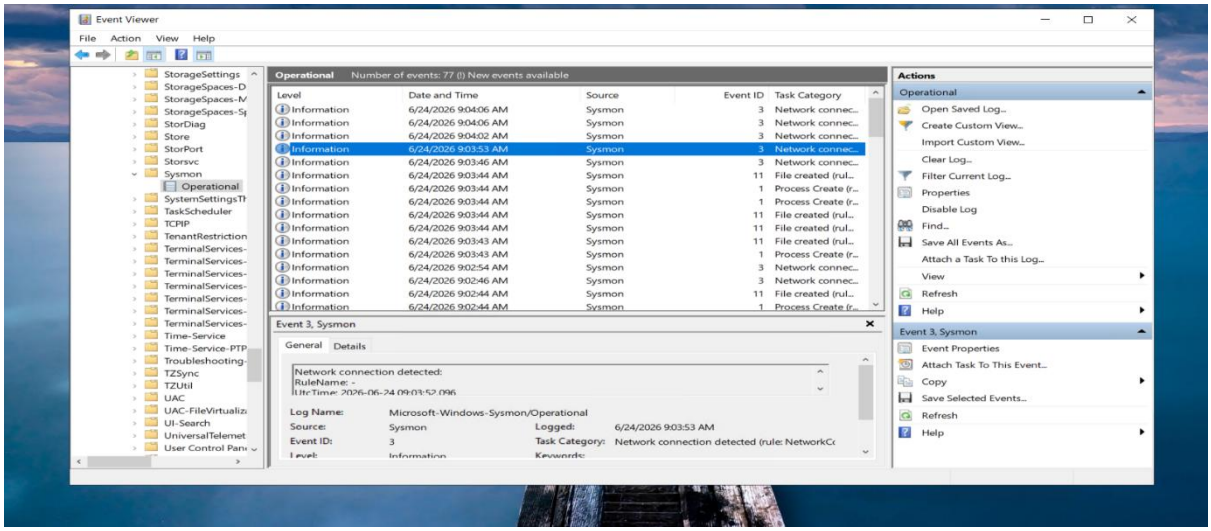

Figure 6.8: Sysmon Logs in Event Viewer.

This step ensures that the system is correctly capturing telemetry data.

6.5 SIEM Workspace Setup.

A Microsoft Sentinel workspace was created to collect and analyze logs.

Setup Steps:

- Login to Microsoft Azure portal.
- Create a Log Analytics Workspace.
- Enable Microsoft Sentinel.
- Configure data connectors.

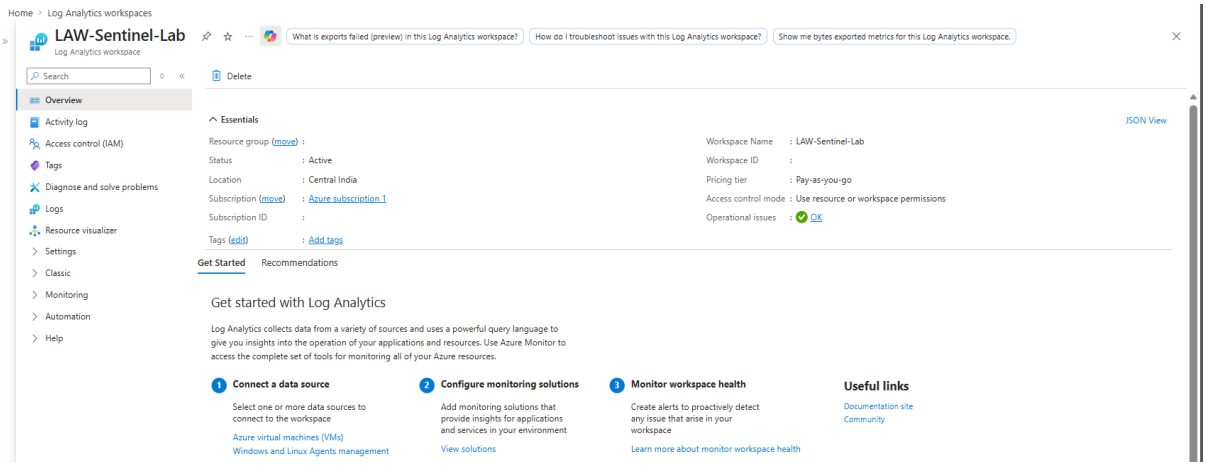

Figure 6.9: Log Analytics Workspace Overview.

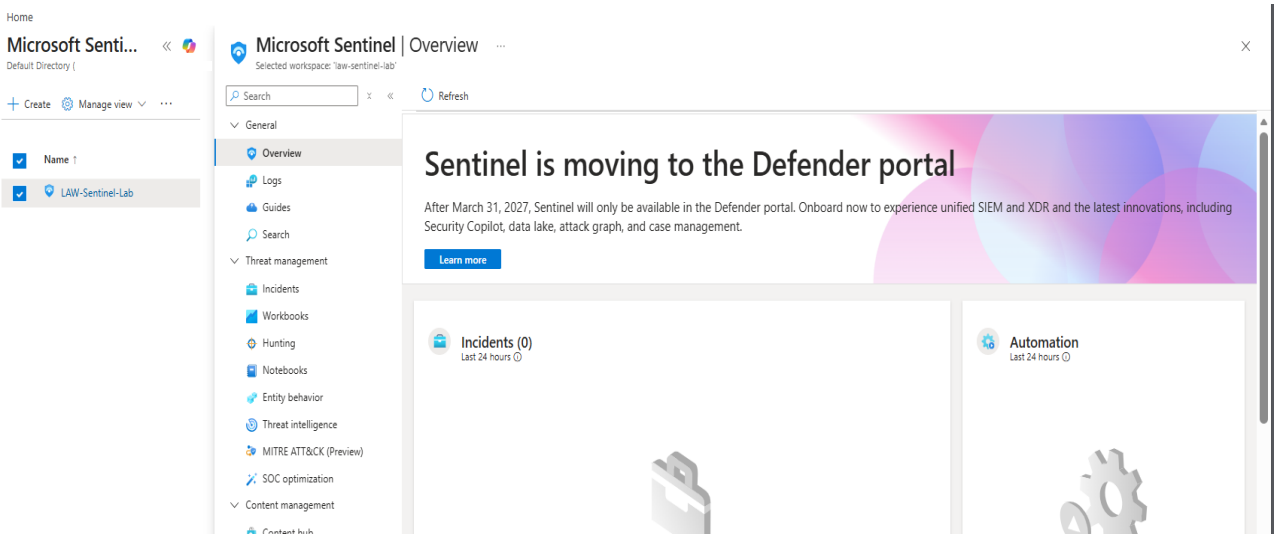

Figure 6.10: Microsoft Sentinel Enabled in Workspace.

This workspace acts as a central platform for storing and analyzing logs collected from endpoints.

6.6 Log Ingestion Process.

The final step in the setup phase is to connect Sysmon logs to Microsoft Sentinel.

Process Overview:

- Configure data connector in Sentinel.
- Link Windows event logs to Sentinel.
- Forward logs from endpoint to cloud workspace.
- Verify logs in Sentinel dashboard.

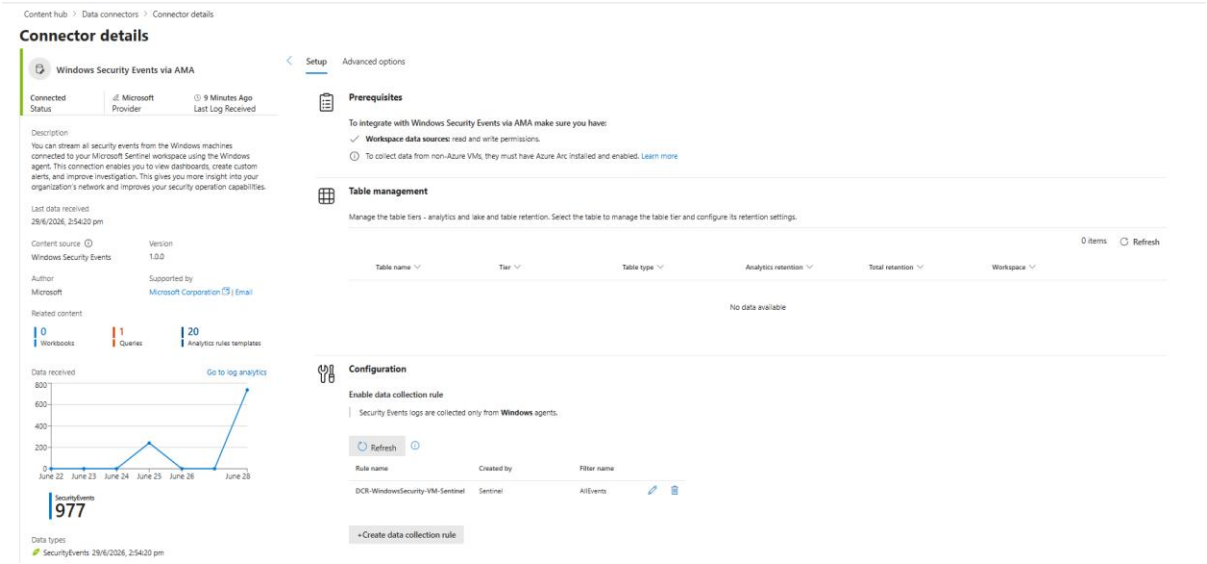

Figure 6.11: Configure Data Connector.

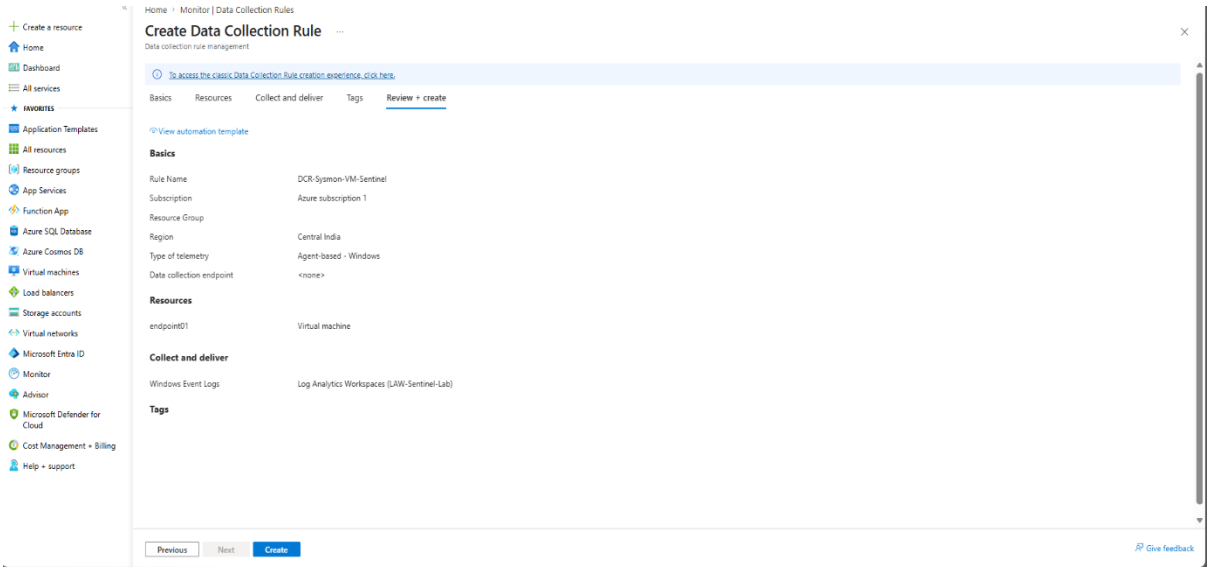

Figure 6.12: Configure Data Collection Rule.

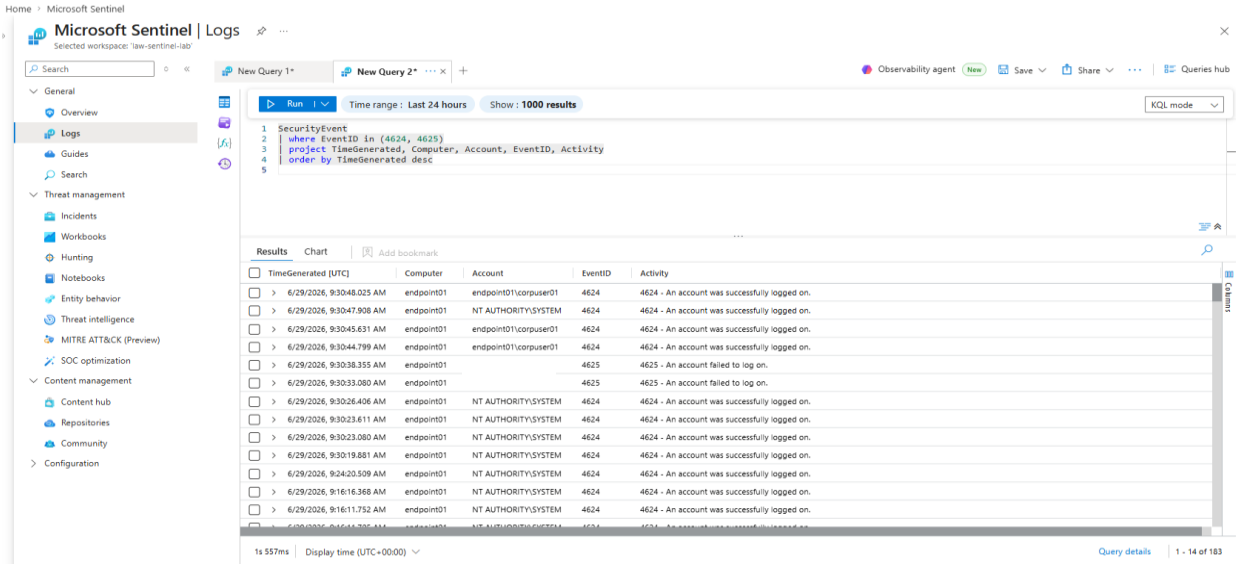

Figure 6.13: Logs in Microsoft Sentinel.

Once the logs are successfully ingested, they can be analyzed using KQL queries for threat detection.

Module 7
DEVELOPMENT PHASE

7.1 Log Collection Process.

The development phase begins with the implementation of log collection from the endpoint system. Sysmon is used to capture detailed system-level events such as process execution, file modifications, and network activity.

Once installed, Sysmon continuously monitors system behavior and generates logs in the Windows Event Log.

These logs include important security-related events such as:

- Process creation events (Event ID 1 for Sysmon).
- Network connection events (Event ID 3).
- File creation events.

Sysmon provides deep visibility into system activity, making it possible to detect malicious behavior patterns. The collected logs form the foundation of the threat detection system.

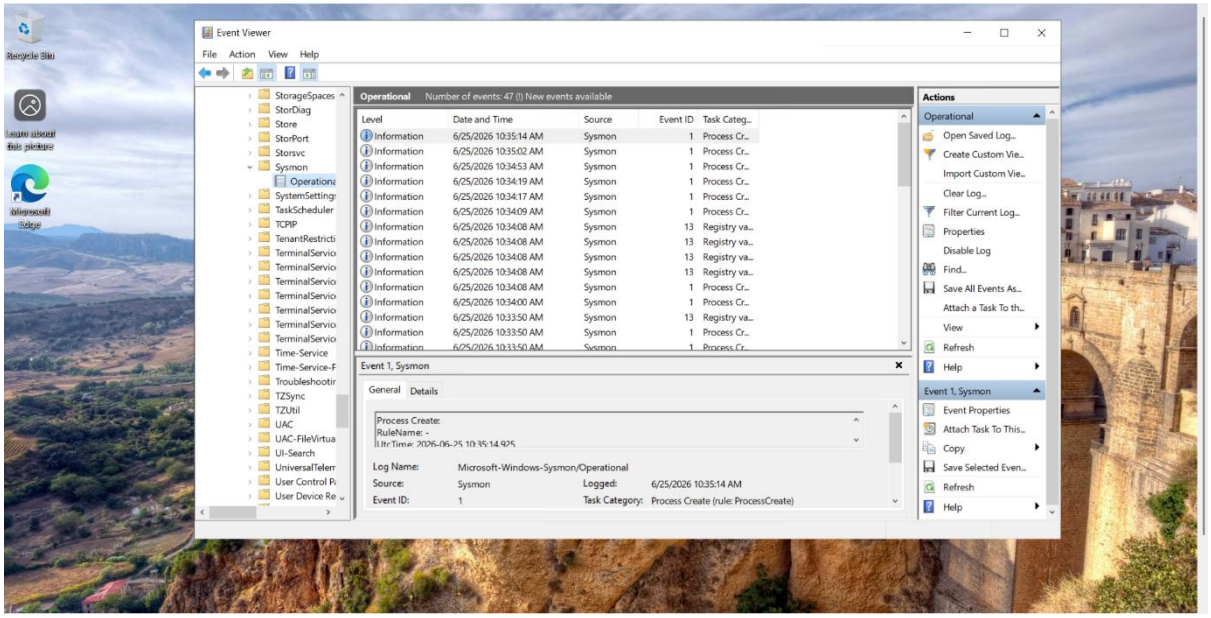

Figure 7.1: Sysmon Logs in Event Viewer.

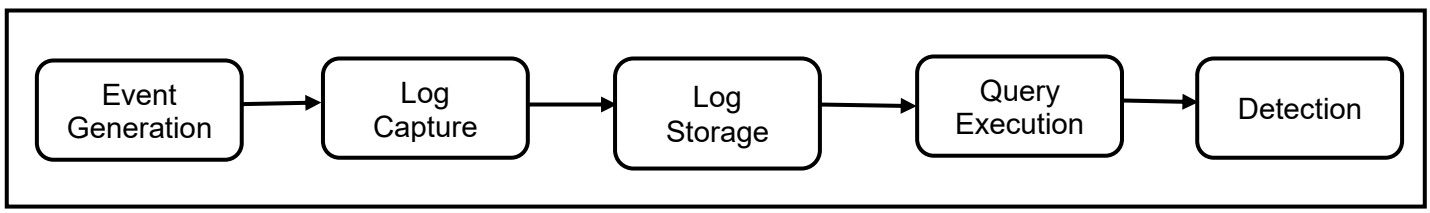

Figure 7.2: Log Processing Workflow.

7.2 Event Monitoring and Analysis.

After log collection, the next step is monitoring and analyzing system events. The logs generated by Sysmon contain detailed information such as:

- Process name and execution path.
- Command-line arguments.
- User account details.
- Parent-child process relationships.

For example, Windows process creation events (Event ID 4688) record every program execution along with user context and command-line details, which are critical for detecting suspicious activity.

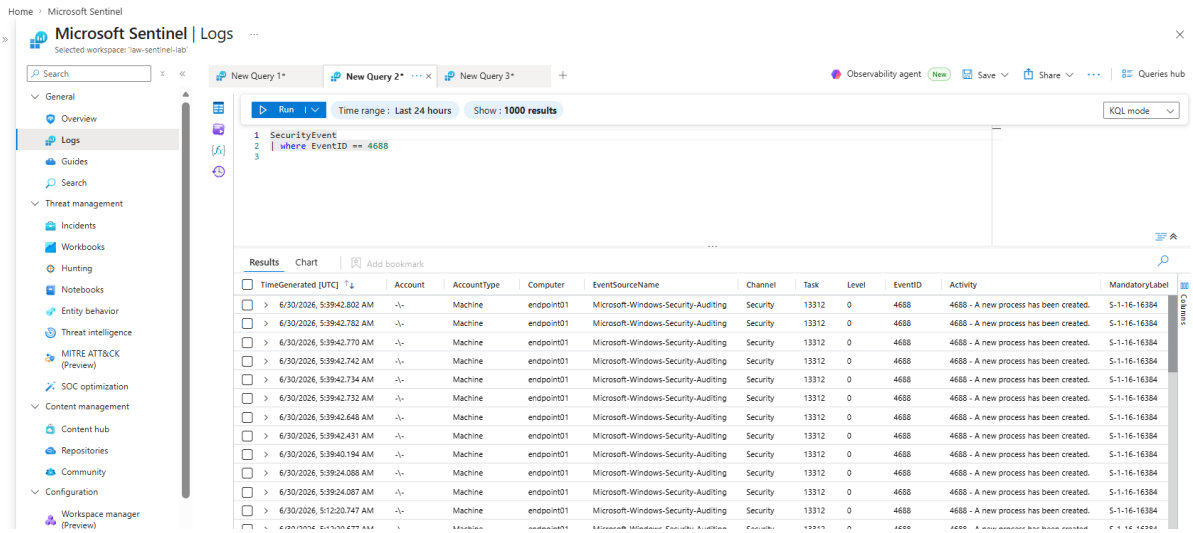

Figure 7.3: Generated Logs in Sentinel Workspace.

Event monitoring is focused on identifying activities such as:

- Execution of unauthorized programs.
- Abnormal use of command-line tools.
- Suspicious parent-child process relationships.

This step ensures that all relevant system behavior is continuously tracked.

7.3 Data Forwarding to SIEM.

Once logs are generated, they are forwarded to Microsoft Sentinel for centralized monitoring and analysis.

The data forwarding process includes:

- Collecting logs from Sysmon.
- Sending logs through an agent or connector.
- Storing logs in a Log Analytics workspace.

In Microsoft Sentinel, all logs are stored centrally and can be queried using KQL. The system uses data collection rules (DCRs) to control how data is ingested and stored in the workspace.

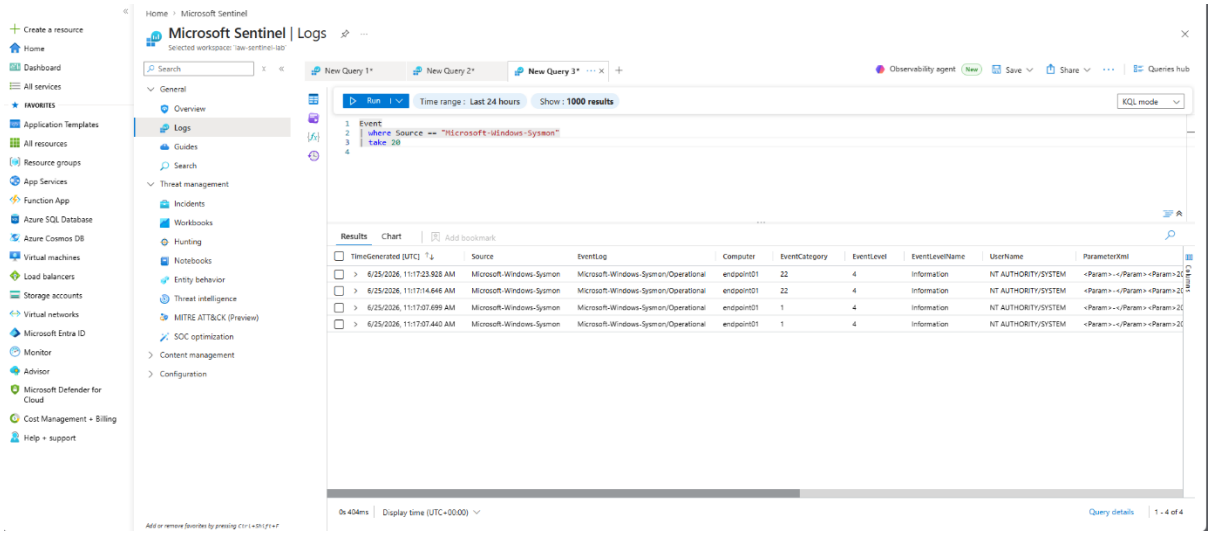

Figure 7.4: Security Events Ingested in Microsoft Sentinel.

This centralized approach allows efficient monitoring of system activities and simplifies analysis.

7.4 Query Development using KQL.

Kusto Query Language (KQL) is used to analyze logs and identify suspicious activities. Queries are written to filter, search, and extract relevant information from large volumes of data.

KQL enables:

- Filtering logs based on conditions.
- Searching for specific activities.
- Identifying abnormal patterns.

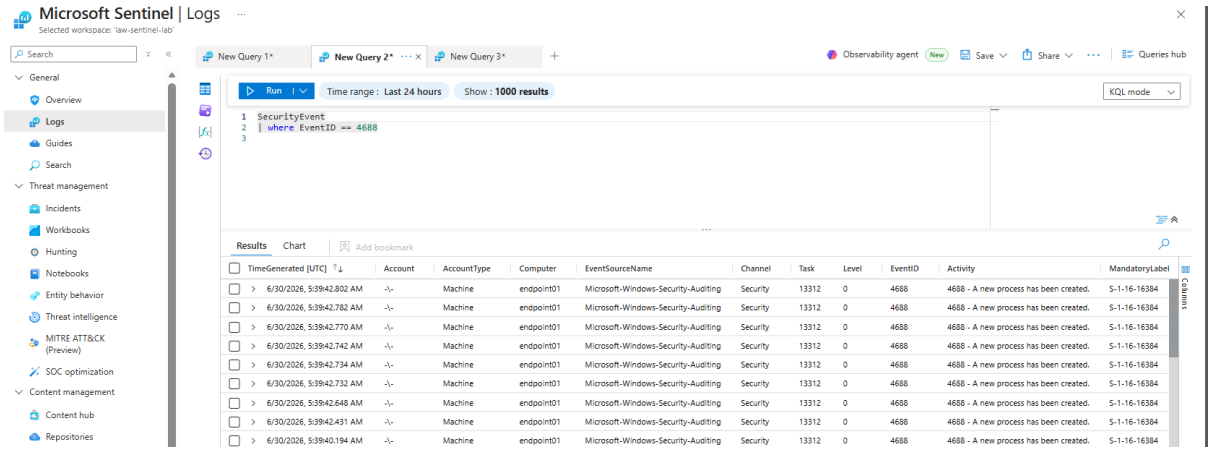

Figure 7.5: KQL Query Page.

7.4.1 Process Monitoring Queries.

These queries are used to monitor process creation events and detect suspicious programs.

Example:

```
SecurityEvent
| where EventID == 4688
| summarize count() by Process, Account
```

This query identifies frequently executed processes and helps detect unusual activity.

7.4.2 Network Activity Queries.

These queries monitor outgoing and incoming network connections.

Example:

```
SysmonEvent
| where EventID == 3
| project Computer, DestinationIp, Image
```

This helps identify suspicious connections or unauthorized communication.

7.4.3 Suspicious Command Detection.

This type of query detects misuse of system tools such as PowerShell.

Example:

```
SecurityEvent
| where EventID == 4688
| where CommandLine contains "powershell"
```

Such queries help identify advanced attacks where legitimate tools are used maliciously.

7.5 Detection Rule Implementation.

Detection rules are created in Microsoft Sentinel based on KQL queries. These rules continuously monitor incoming log data and trigger alerts when suspicious conditions are met.

The detection rules are designed to:

- Identify abnormal process execution.
- Detect suspicious commands.
- Monitor system behavior patterns.

In a SIEM system, detection rules act as automated engines that process logs and generate alerts based on predefined logic.

Rules are configured with parameters such as:

- Query logic.
- Frequency of execution.
- Threshold conditions.

This allows the system to detect threats in real time.

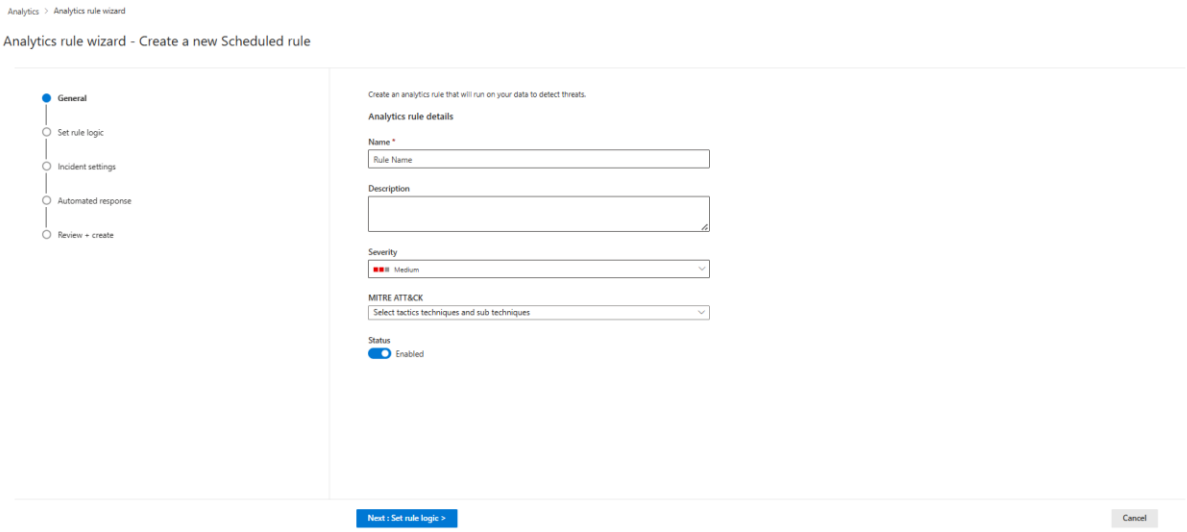

Figure 7.6: Detection Rule Creation.

7.6 Alert Configuration.

Once detection rules are implemented, the system generates alerts when suspicious activities are detected.

Alert configuration includes:

- Defining alert conditions.
- Setting severity levels.
- Linking alerts to detection rules.

When an alert is triggered, it contains information such as:

- Activity details.
- Time of occurrence.
- Affected system.

These alerts help in identifying potential threats and initiating further investigation.

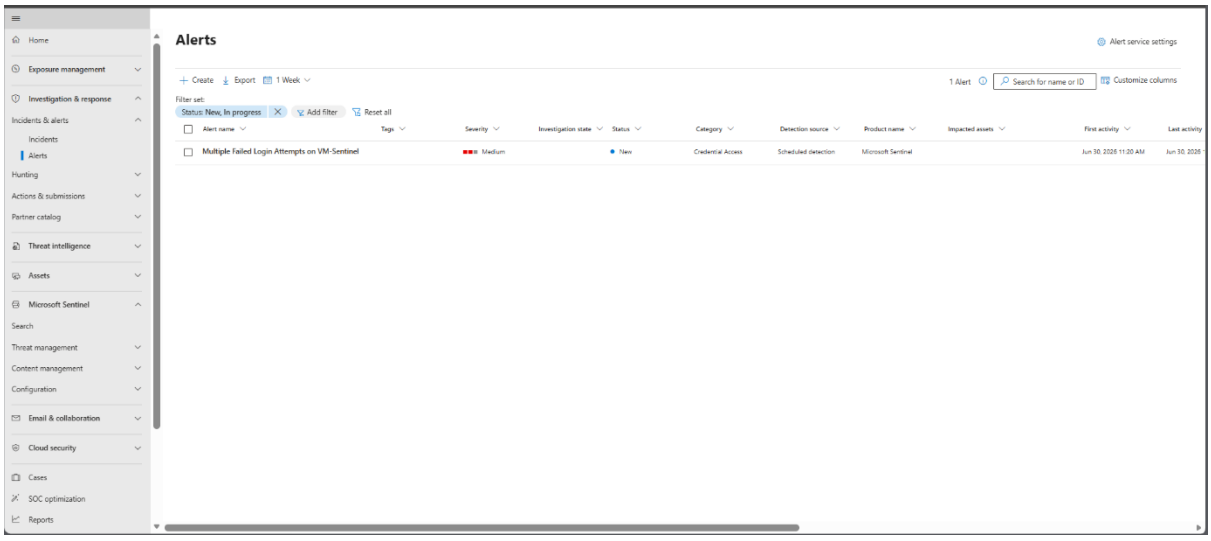

Figure 7.7: Alert generated in Sentinel.

Module 8
IMPLEMENTATION

8.1 System Workflow Implementation.

The implementation phase focuses on integrating all components of the system to achieve real-time threat detection using endpoint telemetry data. The system is implemented in a structured manner by combining log collection, log forwarding, analysis, and alert generation.

The workflow begins with the endpoint system, where user activities generate system events. These events are captured by Sysmon and forwarded to Microsoft Sentinel for analysis. Detection rules are applied to analyze these logs, and alerts are generated when suspicious behavior is identified.

The implemented system ensures:

- Continuous collection of endpoint activities.
- Real-time processing of logs.
- Automated detection using predefined rules.
- Alert generation for security incidents.

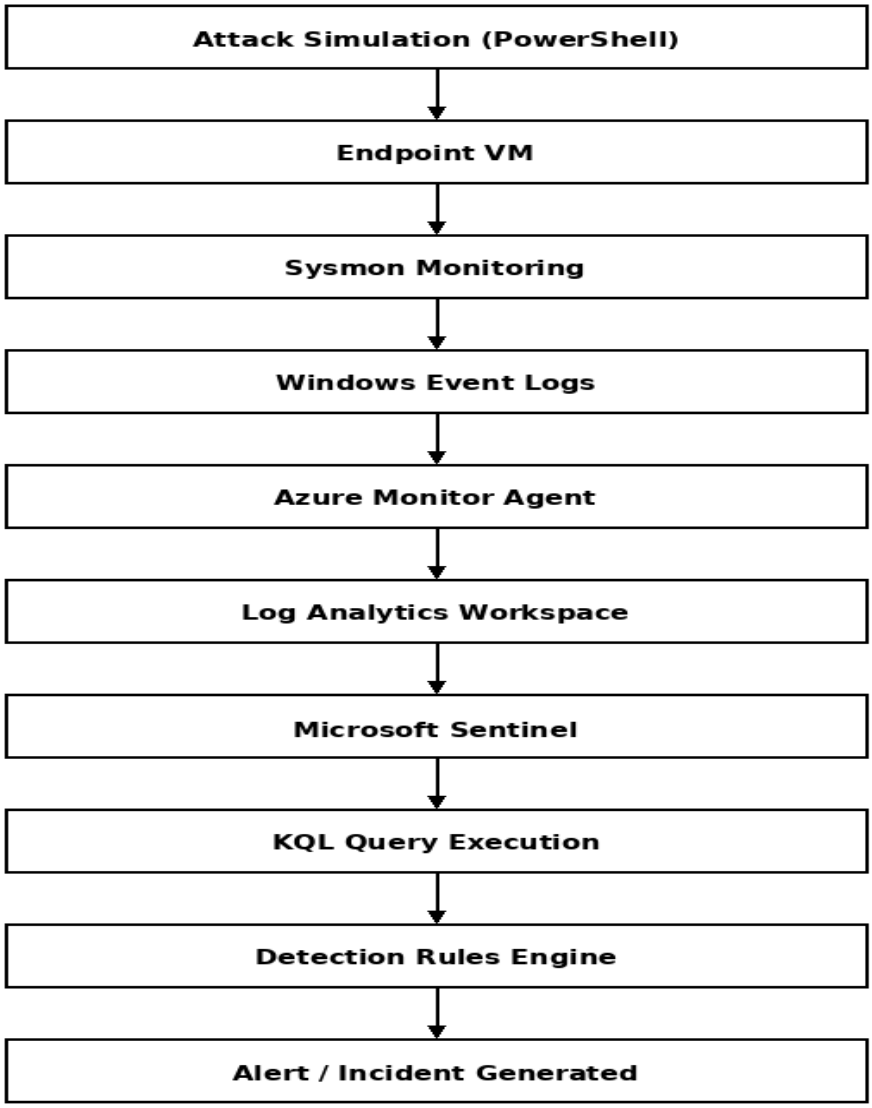

Figure 8.1: Implementation Workflow.

8.2 Module-wise Implementation.

The system is divided into different modules to simplify the implementation and improve system clarity.

8.2.1 Logging Module.

The logging module is responsible for capturing system activity from the endpoint.

This module uses Sysmon to generate logs for:

- Process creation.
- Network activity.
- File changes.

Each system activity is recorded with detailed information such as timestamp, process name, and command-line arguments. These logs are stored in the Windows Event Log. The logging module acts as the data source of the entire system, providing raw telemetry data for analysis.

8.2.2 Detection Module.

The detection module is responsible for analyzing logs and identifying suspicious activities. This module is implemented using Microsoft Sentinel and KQL queries.

Detection rules are created to identify patterns such as:

- Execution of PowerShell commands
- Unusual process behavior
- Suspicious parent-child process relationships

Example logic used:

- Monitor process creation events
- Filter suspicious command executions
- Trigger alerts when conditions are met

This module acts as the brain of the system, converting raw logs into actionable security insights.

8.2.3 Alerting Module.

The alerting module generates alerts based on detection rules triggered in Microsoft Sentinel.

When suspicious activity is detected:

- Alert is generated with event details
- Severity level is assigned
- Log data is stored for further investigation

The alerts provide information such as:

- Time of occurrence
- System affected
- Type of suspicious activity

This module allows quick identification of potential threats and helps in initiating response actions.

8.3 Attack Simulation Setup.

To test the effectiveness of the system, attack scenarios were simulated in the lab environment. These simulations help evaluate how well the system detects suspicious behavior.

Simulated Attacks Include:

- PowerShell execution with encoded commands.
- Suspicious command-line activity.
- Network-based activity.

The attacks were performed in the virtual machine environment to ensure safe testing.

Example simulation:

powershell -enc <encoded command>

Simulation of attacks helps in:

- Validating detection rules.
- Testing system performance.
- Demonstrating real-time monitoring.

8.4 Execution of Test Scenarios.

After setting up the system and simulating attacks, test scenarios were executed to evaluate system performance.

Execution Steps:

- Generate system activity or attack simulation.
- Capture logs using Sysmon.
- Forward logs to Microsoft Sentinel.
- Run detection queries.
- Trigger alert based on condition.

Observed Results:

- Logs were successfully generated and captured.
- Logs were visible in Microsoft Sentinel.
- Detection queries identified suspicious activity.
- Alerts were generated for specific scenarios.

8.5 System Validation.

The system was validated based on its ability to:

- Collect endpoint logs successfully.
- Detect suspicious activities using queries.
- Generate alerts in real time.

The results confirm that the system is capable of performing real-time threat detection using endpoint telemetry.

Module 9
CASE STUDY

9.1 Overview of Attack Scenario.

To assess how effective the proposed threat detection system was, a real-world attack scenario was simulated in a controlled lab environment. The aim of this case study is to examine how different detection techniques respond to suspicious activities and how endpoint telemetry can be utilized to detect threats as they occur.

The chosen attack scenario involves the use of PowerShell execution, which is commonly exploited by attackers for malicious purposes such as downloading payloads, executing scripts, and maintaining persistence. PowerShell-based attacks are hard to identify since they use legitimate system tools.

The system monitors this activity using Sysmon and analyzes logs in Microsoft Sentinel to detect any suspicious behavior.

9.2 PowerShell Attack.

In this experiment, a simulated PowerShell attack was executed on the endpoint system inside the virtual machine.

Attack Steps:

1. Open PowerShell on the endpoint system.
2. Execute a suspicious command (e.g., encoded command execution).
3. System generates logs through Sysmon.
4. Logs are forwarded to Microsoft Sentinel.
5. Detection rules analyze the logs.

Example simulated command:

powershell -enc <encoded command>

This type of command is often used by attackers to hide malicious scripts. The execution of such commands generates process creation logs that can be analyzed for suspicious behavior.

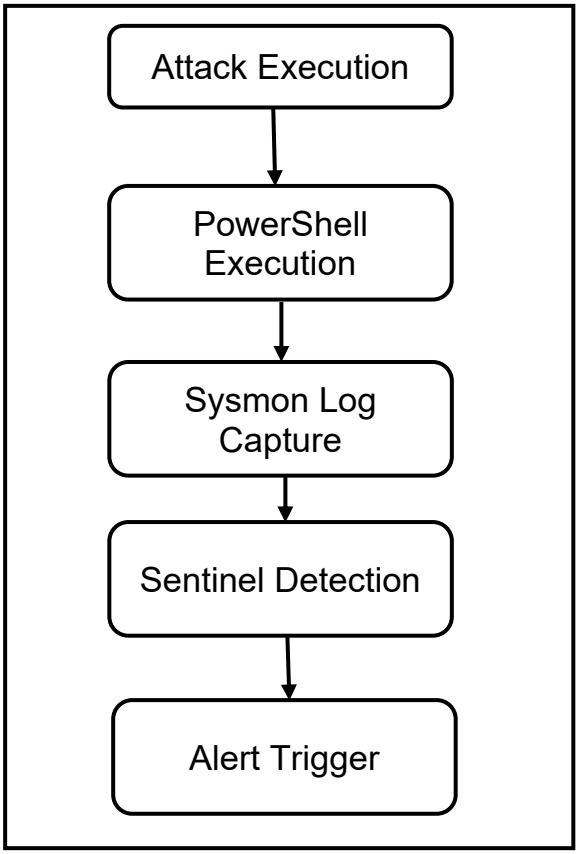

Figure 9.1: Attack Detection Flow.

9.3 Signature-Based Detection Analysis.

Signature-based detection works by identifying known patterns of malicious activity.

Observation in Case Study:

- The simulated attack did not match any predefined signature
- No known malware signature was associated with the command

Detection Result:

- No alert generated
- Attack was not detected

Reason:

Signature-based systems rely on known attack patterns. Since the simulated attack used a custom command, it was not recognized as malicious.

Conclusion:

- This method is not effective against:
- Zero-day attacks
- Custom scripts
- Advanced attacker techniques

9.4 Anomaly-Based Detection Analysis.

Anomaly-based detection identifies deviations from normal system behavior.

Observation in Case Study:

- The system detected PowerShell usage as unusual activity
- The behavior was flagged as deviation from baseline

Detection Result:

- Partial detection
- Alert generated (with low confidence)

Issue Observed:

- High number of false positives
- Normal activities may also be flagged

Conclusion:

Anomaly-based detection is useful for identifying unknown threats but requires proper tuning to reduce false alerts.

9.5 Behavior-Based Detection Analysis.

Behavior-based detection analyzes system activities and identifies suspicious patterns.

Observation in Case Study:

- Sysmon captured process creation details
- Logs contained command-line arguments and execution details
- KQL queries analyzed suspicious activity

Example detection query:

```
SecurityEvent
| where EventID == 4688
| where CommandLine contains "powershell"
```

Process creation logs provide detailed information about execution patterns, including command-line arguments and user context, which are critical for detecting suspicious behavior.

Detection Result:

- Attack successfully detected
- Alert generated in Microsoft Sentinel

Advantages Observed:

- High detection accuracy
- Ability to detect unknown and advanced threats
- Lower false positives compared to anomaly-based detection

9.6 Comparative Results.

The performance of different detection techniques is summarized below:

Detection Technique	Detection Result	Accuracy	False Positives
Signature-Based	Not Detected	High (known only)	Low
Anomaly-Based	Partially Detected	Medium	High
Behavior-Based	Successfully Detected	High	Medium–Low

Table 9.1: Performance Table

9.7 Key Observations.

The following observations were made from the case study:

- Signature-based detection is limited to known threats
- Anomaly-based detection can identify unknown threats but generates noise
- Behavior-based detection provides the best balance between accuracy and reliability
- Endpoint telemetry plays a crucial role in identifying suspicious activities

The use of detailed process logs allows tracking of command execution and helps detect advanced attack techniques more effectively.

9.8 Conclusion of Case Study.

The case study demonstrates that behavior-based detection using endpoint telemetry and SIEM tools is the most effective approach for modern threat detection.

The results validate the proposed system design, where logs collected from Sysmon are analyzed using Microsoft Sentinel to detect suspicious activities in real time. The system successfully identifies malicious behavior and generates alerts, proving its effectiveness in a controlled environment.

Module 10
RESULT AND ANALYSIS

10.1 Log Collection Results.

The first outcome of the implemented system is the successful collection of endpoint telemetry data using Sysmon. The system was able to capture various system-level events, including process execution, network connections, and file activity.

The collected logs were verified through the Windows Event Viewer, where detailed information such as process name, execution path, timestamp, and command-line arguments were recorded. The logs were generated in real time whenever user activity or simulated attacks were performed.

After collection, these logs were successfully forwarded to Microsoft Sentinel, where they were stored in the Log Analytics workspace. This confirms that the log collection and forwarding pipeline is working correctly.

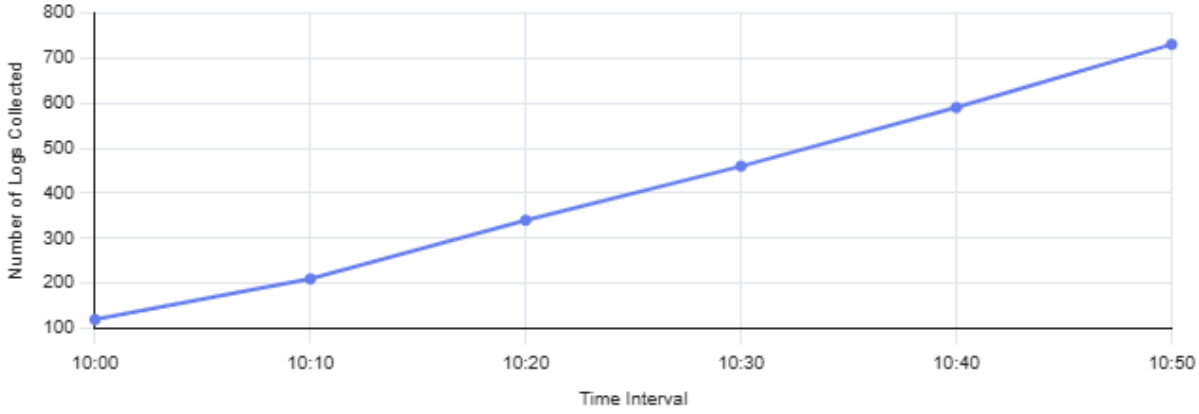

Figure 10.1: Number of Logs Collected Over Time.

10.2 Detection Accuracy.

The detection accuracy of the system was evaluated based on its ability to correctly identify suspicious activities during simulated attack scenarios.

Observations:

- The system successfully detected PowerShell-based attack simulations
- Detection rules identified suspicious command execution patterns
- Alerts were generated with relevant log details

The use of behavior-based detection techniques improved the accuracy by focusing on activity patterns rather than known signatures.

Analysis:

- Behavior-based detection showed high accuracy in detecting malicious activities.
- Signature-based detection failed to detect custom attacks.
- Anomaly-based detection detected some activities but generated false positives.

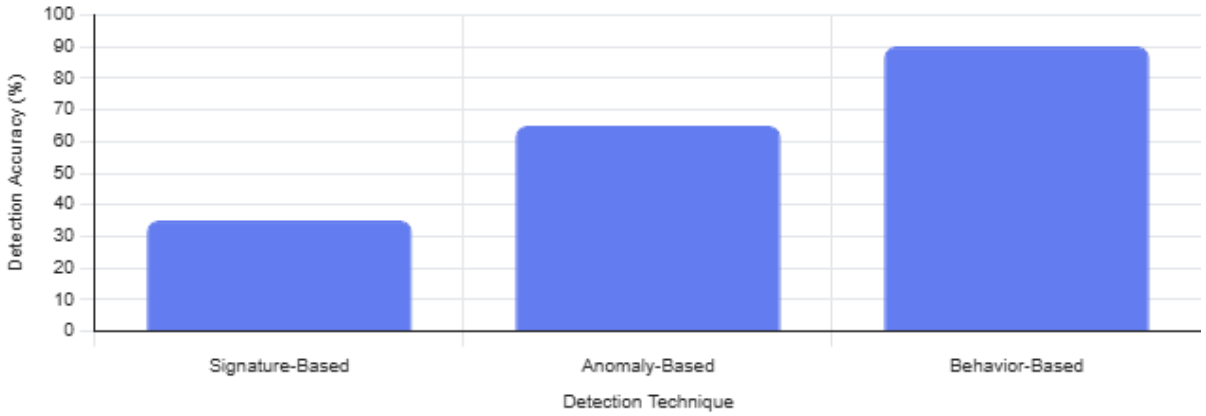

Figure 10.2: Detection Accuracy Comparison.

10.3 Alert Generation Analysis.

The system generated alerts in Microsoft Sentinel whenever detection rules were triggered. Each alert contained relevant details required for analysis.

Alert Information Includes:

- Timestamp of activity
- System or endpoint affected
- Type of suspicious activity
- User associated with the event

The alerts were generated in near real time, which allows quick identification of threats.

Observations:

- Alerts were triggered for suspicious PowerShell commands
- Alerts contained sufficient data for investigation
- Detection rules were able to identify high-risk activities

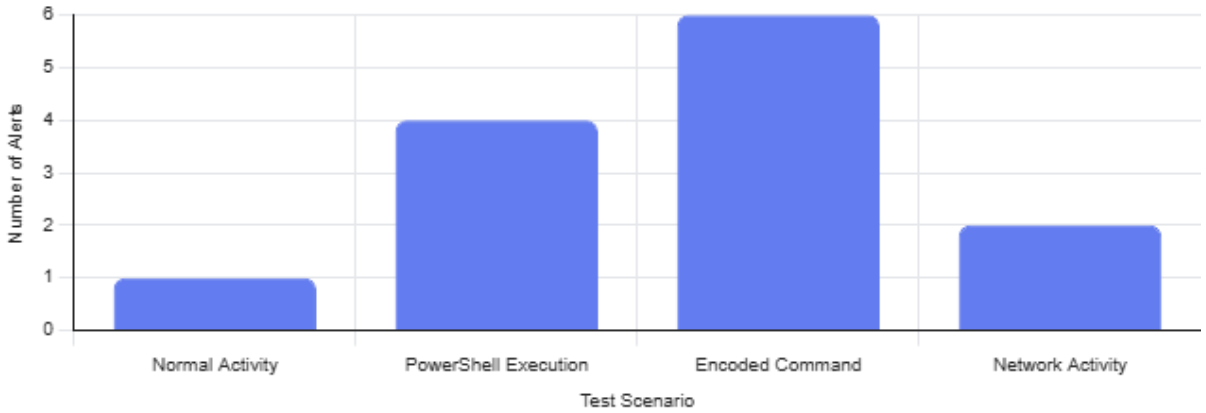

Figure 10.3: Alerts Generated by Test Scenario.

10.4 Performance Evaluation.

The performance of the system was evaluated based on its ability to handle log collection, processing, and detection in real time.

Performance Factors Considered:

- Speed of log collection
- Timeliness of detection
- Response time of alert generation

Results:

- Logs were collected without delay
- Log ingestion into Sentinel was successful
- Detection queries executed efficiently

The system demonstrated stable performance in a controlled lab environment, with minimal delay between activity execution and alert generation.

10.5 False Positives Analysis.

False positives refer to situations where normal system behavior is incorrectly identified as suspicious.

Observations:

- Some normal PowerShell usage triggered alerts
- Certain administrative actions were flagged as suspicious

Analysis:

- Anomaly-based detection generated more false positives
- Behavior-based detection reduced false positives but still required tuning

To minimize false positives:

- Detection rules need further refinement
- Filtering conditions should be improved
- Baseline system activity should be defined

Figure 10.4: False Positives vs Actual Alerts.

10.6 Graphical Representation of Results.

To better understand the system's performance, the results can be represented graphically.

Suggested Graphs:

- Number of logs collected over time.
- Number of alerts generated.
- Detection accuracy comparison.
- False positives vs actual alerts.

10.7 Overall Analysis.

The analysis of results shows that the system is effective in detecting suspicious activities using endpoint telemetry data. The integration of Sysmon and Microsoft Sentinel provides a powerful platform for monitoring and analysis.

Key Findings:

- Endpoint telemetry improves system visibility
- SIEM enables centralized analysis
- Behavior-based detection is the most effective approach
- Real-time monitoring significantly reduces detection time

The system demonstrates practical implementation of threat detection techniques and provides a strong foundation for future enhancements.

Module 11
CHALLENGES AND LIMITATIONS

11.1 Introduction.

During the development and implementation of the real-time threat detection system, several challenges were encountered. These challenges highlight the complexity of building a cybersecurity monitoring system and the limitations of current tools and techniques.

This chapter discusses the technical difficulties faced, system limitations, and factors that affected the overall performance of the project.

11.2 Technical Challenges.

11.2.1 System Configuration Complexity.

One of the main challenges faced was the configuration of tools such as Sysmon and Microsoft Sentinel. Proper configuration is essential to ensure accurate data collection and analysis.

- Sysmon requires configuration files to define what events should be logged.
- Incorrect configuration can lead to missing important logs or generating excessive data

This required careful tuning and testing to achieve optimal results.

11.2.2 Log Integration Issues.

Integrating Sysmon logs with Microsoft Sentinel was a complex process. Ensuring that logs were correctly forwarded and stored in the SIEM required proper configuration of data connectors.

Challenges included:

- Ensuring correct mapping of log fields
- Verifying Log ingestion in the Sentinel Workspace
- Handling data format differences

This step required multiple iterations and validation

11.2.3 Handling Large Volume of Data.

Endpoint telemetry generates a large volume of log data. Managing and analyzing this data efficiently is challenging, especially in real-time systems.

Issues observed:

- High log volume increases processing load
- Difficult to filter relevant events
- Risk of missing important events due to noise

Filtering and optimizing log collection was necessary to address this issue.

11.2.4 Detection Rule Tuning.

Designing effective detection rules was another challenge. Detection rules must be accurate enough to detect threats while avoiding false positives.

Challenges included:

- Defining correct query conditions
- Identifying suspicious patterns
- Avoiding unnecessary alerts

Continuous tuning and testing of rules were required to improve performance.

11.3 Data Handling Issues.

One of the major challenges in the system was the generation of false positives.

Observations:

- Normal system activity was sometimes flagged as suspicious
- PowerShell commands used for legitimate tasks triggered alerts

Impact:

- Increased workload for analysis
- Reduced confidence in detection system

This highlights the need for:

- Better rule refinement
- Understanding of normal system behavior

11.4 Limitations of Current System.

Despite successful implementation, the system has certain limitations that restrict its applicability in real-world enterprise environments.

11.4.1 Limited Lab Environment.

The system was implemented in a controlled lab environment using a single virtual machine.

- Does not represent large-scale enterprise infrastructure
- Limited number of endpoints monitored
- Reduced complexity compared to real environments

11.4.2 Rule-Based Detection Approach.

The detection system is based on rule-based logic using KQL queries.

- Cannot automatically adapt to new attack patterns.
- Depends on manually defined rules.
- Limited capability in detecting unknown threats without predefined logic.

11.4.3 Lack of Automation.

The system does not include automated incident response mechanisms.

- Alerts require manual analysis
- No automatic blocking or mitigation of threats
- Limited response capabilities

11.4.4 Absence of Machine Learning Techniques.

The system does not use advanced techniques such as machine learning or AI-based detection.

- Unable to identify complex behavioral patterns automatically
- Limited predictive capabilities

11.4.5 Scalability Constraints.

The current implementation is not optimized for large-scale deployment.

- Performance may degrade with large data volumes
- Requires additional infrastructure for scaling
- Log storage and cost management are not optimized

11.5 Security and Privacy Considerations.

The use of endpoint telemetry involves collecting detailed system information, which may include sensitive data.

Challenges Include:

- Ensuring secure storage of log data
- Preventing unauthorized access to logs
- Maintaining user privacy

Proper access control and encryption mechanisms are required to address these concerns in real-world implementations.

11.6 Lessons Learned.

During the project, several key lessons were learned:

- Importance of detailed logging for threat detection
- Need for proper configuration and tuning of tools
- Challenges in balancing detection accuracy and false positives
- Role of SIEM systems in centralized monitoring

These insights provide valuable understanding for future improvements.

11.7 Summary.

This chapter discussed the challenges faced during the implementation of the threat detection system and highlighted the limitations of the proposed approach.

Although the system successfully demonstrates real-time threat detection, it is limited by factors such as rule-based detection, lack of scalability, and absence of automation.

Addressing these challenges in future work can significantly improve the effectiveness and usability of the system.

Module 12
FUTURE WORK

12.1 Introduction.

Although the proposed system successfully demonstrates real-time threat detection using endpoint telemetry, there are several areas where the system can be enhanced and extended further. Future improvements can help increase detection accuracy, improve system scalability, and enable automated response capabilities.

This chapter outlines possible enhancements and future developments that can strengthen the system and make it more suitable for real-world enterprise environments.

12.2 Advanced Detection Techniques.

The current system uses rule-based detection methods based on predefined queries. In the future, more advanced detection techniques can be incorporated to improve threat detection capabilities.

Possible Improvements:

- Implementation of behavioral analytics based on attack patterns
- Use of threat intelligence feeds for detecting known attack indicators
- Correlation of events from multiple sources for improved detection

These techniques can help identify complex attack scenarios that cannot be detected using simple rules.

12.3 Machine Learning Integration.

One of the major future enhancements is the integration of machine learning algorithms into the detection system.

Advantages of Machine Learning:

- Ability to learn from historical data
- Detection of unknown or zero-day attacks
- Reduction of false positives
- Identification of hidden patterns in large datasets

Machine learning models can analyze system behavior over time and automatically identify anomalies without relying on predefined rules. This would significantly improve the system's intelligence and adaptability.

12.4 Automation of Incident Response.

Currently, the system generates alerts that require manual analysis. In future implementations, automation can be added to improve response time and reduce human effort.

Possible Enhancements:

- Automatic blocking of suspicious processes.
- Integration with security response tools.
- Creation of automated workflows (SOAR integration).

Automation will enable the system to not only detect threats but also respond to them effectively in real time.

12.5 Improved Dashboard and Visualization.

The current system provides basic alert information through Microsoft Sentinel. Future work can focus on improving data visualization and reporting.

Enhancements Include:

- Creation of interactive dashboards.
- Visualization of attack patterns using graphs.
- Real-time monitoring panels.

Better visualization helps security analysts understand threats more easily and take quick action.

12.6 Scalability for Enterprise Deployment.

The current implementation is limited to a small lab environment. In the future, the system can be expanded for large-scale enterprise deployment.

Areas of Improvement:

- Monitoring multiple endpoints simultaneously
- Handling large volumes of log data
- Optimizing system performance and storage

Scalability improvements will make the system more practical for real-world use.

12.7 Integration with Additional Data Sources.

The system currently focuses on endpoint telemetry. Future enhancements can include integration with additional data sources.

Examples:

- Network logs from firewalls
- Cloud activity logs
- Identity and access management logs

Combining multiple data sources will provide a more comprehensive view of system activity and improve detection accuracy.

12.8 Reduction of False Positives.

False positives are a common challenge in detection systems. Future improvements can focus on refining detection rules and improving accuracy.

Approaches Include:

- Fine-tuning detection queries
- Creating baseline behavior models
- Applying filtering techniques

Reducing false positives will increase system reliability and reduce workload on security teams.

12.9 Cloud Security Enhancements.

Since modern enterprise systems are moving towards cloud environments, future work can focus on extending the system for cloud security.

Enhancements Include:

- Monitoring cloud-based applications
- Detecting suspicious cloud activities
- Integration with cloud security tools

This will help in providing a comprehensive security solution across both on-premises and cloud environments.

12.10 Summary.

This chapter discusses various improvements and enhancements that can be implemented in the future to strengthen the proposed system.

Key areas of improvement include:

- Advanced detection techniques
- Machine learning integration
- Automated incident response
- Improved visualization and scalability

These enhancements will help transform the current prototype into a more robust and enterprise-ready cybersecurity solution.

Module 13
CONCLUSION

13.1 Introduction.

This dissertation focused on the design and implementation of a real-time threat detection system using endpoint telemetry in enterprise environments. With the increasing complexity of cyber threats, traditional security mechanisms are no longer sufficient to detect advanced attacks. This project addressed the need for improved visibility and real-time monitoring using modern cybersecurity tools.

The study explored how endpoint telemetry, when combined with a SIEM solution, can enhance threat detection capabilities and improve incident analysis in a controlled environment.

13.2 Summary of Work.

The project was carried out in multiple phases, starting from understanding the problem, reviewing existing technologies, and designing a practical solution.

Key steps involved in the project include:

- Studying the cybersecurity threat landscape and identifying key challenges
- Understanding the role of endpoint telemetry in threat detection
- Designing a system architecture using Sysmon and Microsoft Sentinel
- Setting up a lab environment using a virtual machine
- Collecting and analyzing endpoint logs
- Implementing detection rules using KQL queries
- Simulating real-world attack scenarios
- Evaluating system performance through results and analysis

Each phase contributed to building a functional system capable of detecting suspicious activities in real time.

13.3 Key Findings.

The results of the project highlight several important findings:

- Endpoint telemetry provides detailed visibility into system activity
- SIEM platforms enable centralized monitoring and analysis of logs
- Behavior-based detection is more effective than traditional signature-based methods
- Real-time log analysis helps in early identification of threats
- Proper configuration and tuning of detection rules is essential for accurate results

The case study conducted in this project demonstrated that suspicious PowerShell activities can be successfully detected using telemetry data and SIEM-based analysis.

13.4 Achievements of the Project.

The following achievements were accomplished during the project:

- Successful implementation of a real-time threat detection system
- Integration of Sysmon with Microsoft Sentinel
- Development of detection rules using KQL queries
- Generation of alerts for suspicious activities
- Simulation and analysis of real-world attack scenarios

The system effectively demonstrated how enterprise security monitoring can be performed using endpoint telemetry.

13.5 Overall Conclusion.

From the results and analysis, it can be concluded that the proposed system is effective in detecting suspicious activities using endpoint telemetry data. The integration of Sysmon and Microsoft Sentinel provides a powerful approach for real-time monitoring and threat detection.

Although the system is implemented in a controlled environment, it reflects the working principles of real-world Security Operations Centers (SOC). The use of behavior-based detection improves the ability to identify advanced threats, which are difficult to detect using traditional methods.

The project successfully meets its objectives and demonstrates the importance of combining endpoint monitoring with SIEM tools for effective cybersecurity.

13.6 Final Remarks.

This dissertation provides a practical understanding of how modern cybersecurity systems operate in real-world environments. The knowledge gained from this project can be applied to enterprise security monitoring, threat hunting, and incident response.

The study also highlights the need for continuous improvement in detection techniques to keep up with evolving cyber threats.

Overall, the project serves as a foundation for further research and development in the field of cybersecurity and real-time threat detection.

FUTURE PLAN:

S. No.	Phases	Start Date – End Date	Work to be Done	Status
1.	Dissertation Outline	Week 1 – Week 2	Literature review and prepare Dissertation outline.	Completed
2.	Setup Phase	Week 3 – Week 4	Design Activity.	Completed
3.	Development Phase	Week 5 – Week 8	Development Activity.	Completed
4.	Testing	Week 9 – Week 11	Deployment Testing, User Evaluation and Conclusion.	Pending
5.	Dissertation Review	Week 12 – Week 13	Submit dissertation to Supervisor & Additional Examiner for review and feedback.	Pending
6.	Submission	Week 14 – Week 15	Final review and submission of dissertation.	Pending

ABBREVIATIONS:

Abbreviation	Full Form
SIEM	Security Information and Event Management
VM	Virtual Machine
KQL	Kusto Query Language
OS	Operating System
DFD	Data Flow Diagram
EDR	Endpoint Detection and Response
SOC	Security Operations Centre
CMD	Command Prompt
RAM	Random Access Memory
CPU	Central Processing Unit
GB	Gigabyte
IT	Information Technology
AI & ML	Artificial Intelligence & Machine Learning
DCR	Data Collection Rule
API	Application Programming Interface
RDP	Remote Desktop Protocol